

INTERNAL ENGINEERING REFERENCE · MEDICAL IMAGING SOFTWARE

DICOM Pixel Data Transformation & Rendering Pipeline

A Technical Reference Manual for Engineers Building
DICOM Viewers, PACS, VNA, and Clinical Visualization Software

From stored pixel bytes to the displayed pixel: Modality LUT · VOI LUT · Presentation LUT ·
Photometric Interpretation · Palette Color · Spatial Transforms · GSDF · Cornerstone / OHIF Architecture

Table of Contents

PART I — FOUNDATIONS

- 1. Introduction and the Complete Rendering Pipeline
- 2. The DICOM Image Pixel Module
- 3. Raw Pixel Data at the Bit and Byte Level
- 4. Photometric Interpretation

PART II — THE GRAYSCALE PIPELINE

- 5. Modality LUT: Rescale and Explicit LUTs
- 6. How a Non-Linear LUT Actually Works
- 7. VOI LUT and Windowing
- 8. Presentation LUT and the Grayscale Standard Display Function
- 9. Masking and Subtraction

PART III — COLOR AND GEOMETRY

- 10. Palette Color
- 11. RGB Pixel Data
- 12. Spatial Transformations
- 13. Shutters, Overlays, Annotations, and Presentation States

PART IV — SYSTEMS AND PRACTICE

- 14. Cornerstone, OHIF, and Framework Architecture
- 15. Clinical Safety: Why Pixel Transformation Is a Safety Concern
- 16. End-to-End Worked Case Studies
- 17. Mistakes That Look Reasonable but Are Wrong
- 18. Engineering Architecture and Performance
- 19. Testing and Validation Methodology

PART V — REFERENCE

- 20. The Engineer's Mental Model (Decision Flow)
- 21. Glossary of Terms
- 22. References and Further Reading

1 Introduction and the Complete Rendering Pipeline

This manual assumes you already know what a DICOM dataset, SOP Class, Transfer Syntax, and Pixel Data element are. Its purpose is to take you from that baseline to implementation-level fluency in the transformations that convert stored pixel samples into the image a radiologist actually looks at — and to make you dangerous in code review, because most rendering bugs in imaging software are pipeline-ordering bugs, not math bugs.

1.1 The central discipline: never collapse the stages

Everything in this manual follows from one discipline: **a pixel passes through a strict sequence of named, semantically distinct value spaces**, and an engineer must always be able to say which space a given number lives in. The stages are:

```
Encoded bytes (Transfer Syntax)
→ Stored Value (decoded sample, Image Pixel Module semantics)
→ Modality Value (after Modality LUT / Rescale Slope&Intercept)
→ VOI-transformed Value (after windowing or VOI LUT)
→ Presentation Value / P-Value (after Presentation LUT)
→ Display Driving Level (after GSDF / display calibration)
→ Luminance on the physical display
```

Every one of these is a different variable with a different valid range, a different unit (or no unit at all), and a different owner in the standard. A "pixel value of 40" means nothing until you say which stage you are talking about — a stored value of 40, an HU of 40, and a P-Value of 40 are three unrelated numbers that happen to share a digit string. Bugs happen almost exclusively at the seams: code that reads a stored value and treats it as an HU, code that applies a window to a P-Value, code that writes a VOI-adjusted value back into pixel data. This manual names every seam explicitly so that you can recognize when code is crossing one without the required transformation.

1.2 The formal transformation pipeline

DICOM PS3.3 (Image Pixel Module, VOI LUT Module, Modality LUT Module) and PS3.4 (Grayscale Standard Display Function) define the grayscale pipeline formally as a composition of independent transformations. The figure below is the canonical ordering for a grayscale image with an optional mask and an optional pseudo-color overlay. Color (RGB / true-color) pixel data joins the pipeline downstream, bypassing the Modality/VOI stages, and Palette Color joins after VOI via a dedicated lookup.

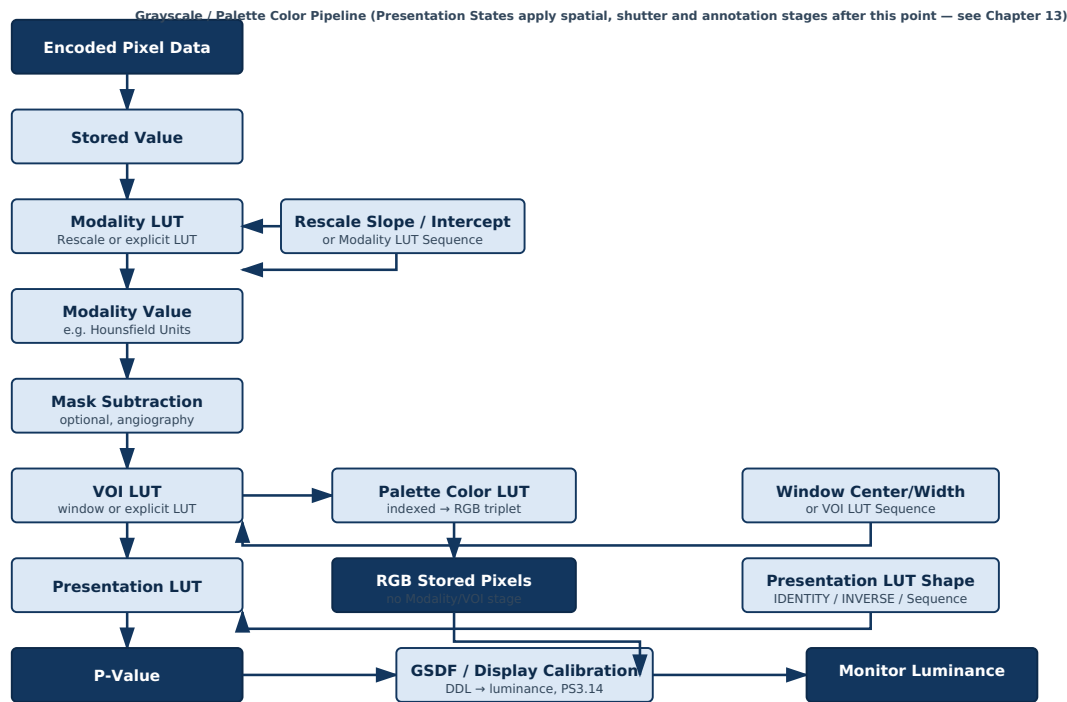


Figure 1.1. The canonical DICOM grayscale rendering pipeline. Each stage is a distinct, independently controllable transformation with its own DICOM attributes. RGB pixel data (bottom branch) enters the display path without a Modality or VOI stage but is still subject to Presentation LUT / GSDF display mapping in some presentation-state contexts.

Which stages are mandatory versus conditional

Stage	Mandatory?	Notes
Stored Value decoding	Always	Governed by Image Pixel Module; not optional under any circumstance.
Modality LUT	Conditionally mandatory	Required for MONOCHROME1/MONOCHROME2 images per the Modality LUT Module; defaults to identity (slope 1, intercept 0) only if truly absent, which is itself non-conformant for most SOP Classes that require Rescale Slope/Intercept.
Mask subtraction	Optional	Only present for X-Ray Angiographic-family images with a Mask Module.
VOI LUT	Conditional, but a viewer must always apply <i>something</i>	If absent, a viewer must derive a default (e.g. full dynamic range windowing); DICOM does not mandate a fallback algorithm, so this is an implementation decision that must be documented.
Presentation LUT	Optional; defaults to IDENTITY (or effectively LIN OD in some contexts)	Absence does not mean "skip" — it means "identity mapping," which is a different statement.
GSDF calibration	Recommended, display-hardware dependent	Not a per-pixel LUT baked into the DICOM object in the general case; a property of the display chain.
Palette Color LUT	Mandatory for PALETTE COLOR	Alternative branch, mutually exclusive with the grayscale VOI/Presentation path for the color-producing step.
Spatial / shutter / overlay / annotation	Conditional	Layered on top of the pixel-value pipeline; do not confuse with pixel-value transformations (Chapter 13).

Where LUTs may be composed, and where they must not be reordered

Because the Modality LUT and a linear VOI window are both affine (or piecewise-affine at the boundaries) operations on a continuous domain, an engine that only needs the final windowed grayscale output *may* mathematically pre-compose Rescale Slope/Intercept with a linear VOI window into a single combined LUT for

performance, provided the composed LUT is rebuilt any time either input changes. This is the basis for GPU shader implementations that apply "one LUT" per frame (Chapter 18). However, composition is only safe when:

- Both stages being merged are actually linear (or you correctly compose a linear stage with a table-driven stage by resampling the table, not by algebraically merging slopes).
- No stage in between depends on the intermediate value for anything other than the next transformation — for example, mask subtraction must happen on Modality Values, not on stored values or VOI output, so it cannot be folded into a pre-composed Stored→P-Value LUT if subtraction is present.
- The Presentation LUT is not merged with VOI when Presentation LUT Shape is INVERSE and the application also needs to expose the un-inverted P-Value for measurement or independent MONOCHROME1 handling.

Conversely, the Modality LUT and VOI LUT must never be reordered: applying a window to a raw stored value and only afterward applying Rescale Slope/Intercept produces a different, clinically wrong image whenever the intercept is non-zero — which for CT (intercept typically -1024) it always is. This single reordering mistake is, in practice, one of the most common defects found in early-stage or hobbyist DICOM viewers, and it is covered in depth with a full worked failure case in Chapter 15.

1.3 Simplified engineering pipeline

The formal model above is what the standard specifies. In an actual codebase you will usually implement something closer to the following, where the standard's independent stages are grouped into cohesive software components:

```
DICOM Reader → Transfer Syntax Decoder → Pixel Sample Decoder
→ Photometric Interpreter (branch: grayscale vs. color vs. palette)
→ Modality Transform → (optional Mask Subtract) → VOI Transform
→ Presentation Transform → Color Conversion (palette/YBR→RGB if needed)
→ Spatial Transform → Overlay/Annotation Compositing → Rendering Backend → GPU/Display
```

This is expanded fully as a software architecture in Chapter 18. The important discipline carried from the formal model is that each box above should correspond to a unit-testable function with a documented input space and output space — never a single monolithic "render()" function that inlines all of the arithmetic, because that is precisely the structure in which ordering bugs hide.

HOW TO USE THIS MANUAL

Each major concept below is explained at three levels, marked with colored tags: **L1 CONCEPTUAL** the mental model, **L2 MATH** the exact numerical transformation, and **L3 ENGINEERING/CLINICAL** implementation consequences and clinical stakes. Worked examples are boxed and numbered per chapter. Read Chapters 1–9 in order — they build on each other. Chapters 10–19 can be read in any order once Part II is understood.

1.4 Nine terminology distinctions to internalize before continuing

Distinction	Why it matters
Stored value $\neq$ physical value	A stored value of 1200 is meaningless without Modality LUT context; it is not itself an HU, a density, or a count.
Modality value $\neq$ display value	An HU of 40 is a physical/clinical quantity; it is not what gets sent to the screen.
VOI transform $\neq$ Modality transform	VOI is display windowing; Modality is unit conversion. Confusing them corrupts both diagnosis and display.
Presentation LUT $\neq$ VOI LUT	Presentation LUT controls polarity/perceptual mapping after windowing, not contrast selection.
PALETTE COLOR $\neq$ RGB	Palette Color is single-sample indexed color; RGB is three independent samples. Treating one as the other corrupts every pixel.
Pixel-array orientation $\neq$ patient-space orientation	Flipping the array is not the same as correctly labeling left/right in patient space.
Pixel Data $\neq$ Presentation State	Pixel Data is the measurement; a Presentation State is one particular way of looking at it.
Display transformation $\neq$ modification of source data	Changing window/level must never rewrite the underlying Pixel Data element.
Framework capability $\neq$ DICOM requirement	What Cornerstone or OHIF happens to do is an implementation choice, not a normative requirement (Chapter 14).

2 The DICOM Image Pixel Module

The Image Pixel Module (PS3.3 C.7.6.3) is the contract that tells you how to turn a byte stream into an array of numbers. Get any one of its attributes wrong and every downstream transformation operates on garbage, even if the Modality/VOI/Presentation math is implemented perfectly.

L1 Think of the Image Pixel Module as a struct definition for the Pixel Data element. Pixel Data itself is an opaque byte blob (OB/OW); the Image Pixel Module attributes are the only thing that tells a reader how those bytes are organized into rows, columns, samples, and bit widths. A decoder that ignores any single attribute in this module is not implementing a partial reading of DICOM — it is implementing a guess.

2.1 Attribute-by-attribute reference

Samples per Pixel (0028,0002)

What it means: the number of separate color/intensity components stored per pixel — 1 for MONOCHROME1/MONOCHROME2/PALETTE COLOR, 3 for RGB and YBR-family interpretations.

Why it exists: the decoder cannot infer this from Photometric Interpretation alone in a fully generic way; the standard makes it explicit so parsers do not need a hardcoded interpretation table for every possible photometric string, and so future photometric interpretations remain parseable by generic code.

How it changes interpretation: it multiplies the number of bytes/samples per pixel position, and determines whether Planar Configuration is meaningful (it is meaningless, and must be absent or ignored, when Samples per Pixel = 1).

Common implementation mistake: hardcoding "1 sample per pixel" for anything read as "grayscale," then silently misreading a PALETTE COLOR image's true per-pixel sample count when a downstream stage assumes RGB-shaped output has already been produced (Palette Color is stored as Samples per Pixel = 1, index values, with the RGB expansion happening only after LUT application — not before).

Photometric Interpretation (0028,0004)

What it means: the semantic meaning of the stored sample(s) — declares MONOCHROME1, MONOCHROME2, PALETTE COLOR, RGB, YBR_FULL, YBR_FULL_422, YBR_PARTIAL_420, YBR_RCT, YBR_ICT, and a small number of others.

Why it exists: the numeric sample values are ambiguous without it — the same 8-bit value 200 means "near-white" in MONOCHROME2, "near-black" in MONOCHROME1, an index into an arbitrary palette in PALETTE COLOR, and one channel of a color triplet in RGB/YBR.

Common implementation mistake: treating Photometric Interpretation as a hint rather than a hard branch condition; see Chapter 4 for the full treatment.

Rows (0028,0010) and Columns (0028,0011)

What they mean: the pixel matrix dimensions, Rows = vertical extent, Columns = horizontal extent.

Common implementation mistake: swapping Rows/Columns when computing stride, which for non-square images produces a diagonally-sheared or truncated image rather than an obviously broken one — a dangerous failure mode because it can look plausible at a glance.

Bits Allocated (0028,0100), Bits Stored (0028,0101), High Bit (0028,0102)

What they mean, as a system (detailed fully with worked binary examples in Chapter 3): Bits Allocated is the physical container size per sample (8 or 16, almost always), Bits Stored is how many of those bits carry meaningful data, and High Bit is the bit position of the most significant meaningful bit within the container (in DICOM's native encoding, High Bit = Bits Stored - 1, i.e., the meaningful bits are right-justified within the allocated container).

Why they exist as three separate attributes: real acquisition hardware frequently produces sample precision that does not align to a byte boundary — 10-bit and 12-bit detectors are common in CR/DR and CT — and storing 12 meaningful bits inside a 16-bit container is far more interoperable than inventing 12-bit-packed byte layouts. Bits Stored also directly defines the maximum representable dynamic range, which downstream code needs to correctly interpret Window Width defaults, LUT domains, and clipping behavior.

Common implementation mistake: assuming Bits Allocated == Bits Stored (i.e., assuming all 16 allocated bits are meaningful) — this silently corrupts windowing and LUT indexing for any 10- or 12-bit modality data, because the unused high bits, if not masked, can be misread as meaningful magnitude.

Pixel Representation (0028,0103)

What it means: 0 = unsigned integer samples, 1 = signed integer (two's complement) samples.

Why it exists: some modalities (post-Rescale CT stored directly as signed, some MR sequences, subtraction images) produce data that is naturally signed at the stored-value level, not only after the Modality LUT.

Common implementation mistake: failing to sign-extend correctly when Bits Stored < Bits Allocated for signed data — the sign bit lives at High Bit, not at bit 15 of the container, so a naive `int16_t` cast on 12-bit signed data with unmasked upper bits produces wrong values (full worked example in Chapter 3.3).

Planar Configuration (0028,0006)

What it means: for Samples per Pixel > 1, whether samples are stored interleaved per pixel (0, e.g. RGBRGBRGB...) or planar, one full component plane at a time (1, e.g. RRRR...GGGG...BBBB...). Meaningless and absent for Samples per Pixel = 1.

Common implementation mistake: hardcoding interleaved layout because it is the more common convention in consumer image formats, silently producing a scrambled or three-banded image when Planar Configuration = 1 is encountered (full byte-level treatment in Chapter 11).

Number of Frames (0028,0008)

What it means: declares a multi-frame Pixel Data element containing N stacked frames of identical geometry, addressed contiguously (native) or via per-frame items/offset tables (encapsulated, Enhanced multi-frame IODs).

Common implementation mistake: assuming single-frame stride arithmetic and reading only the first frame, or assuming frames are always geometrically and photometrically homogeneous when Enhanced Multi-frame per-frame functional groups actually vary VOI, orientation, or even Photometric Interpretation-adjacent parameters frame-to-frame.

Pixel Aspect Ratio (0028,0034) and Pixel Spacing (0028,0030)

What they mean: Pixel Aspect Ratio (vertical/horizontal ratio, integer pair) applies when pixels are not square and Pixel Spacing is absent; Pixel Spacing gives the physical row and column spacing in millimeters and takes precedence when present. These control aspect-correct display and any measurement tooling, not pixel value interpretation.

Common implementation mistake: rendering images 1:1 in device pixels regardless of Pixel Aspect Ratio, distorting apparent anatomy proportions (most visible on older ultrasound and some nuclear medicine images with non-square pixels).

Smallest/Largest Image Pixel Value (0028,0106 / 0028,0107)

What they mean: optional informational attributes giving the actual min/max stored sample values present in the image — useful as a fast hint for default windowing, but not authoritative and not a substitute for reading actual Bits Stored-defined range or for deriving a real default VOI.

Common implementation mistake: trusting these values as the true data range without validating against Bits Stored, since they are optional, occasionally absent, and occasionally stale relative to the actual pixel data (e.g. after some pixel processing steps that don't recompute them).

Pixel Data (7FE0,0010), Float Pixel Data (7FE0,0008), Double Float Pixel Data (7FE0,0009)

What they mean: the sample container itself. Pixel Data (OB or OW, per Bits Allocated and Transfer Syntax) covers the overwhelming majority of images and is subject to the integer bit-packing rules of Chapter 3. Float Pixel Data (32-bit IEEE 754 float, OF) and Double Float Pixel Data (64-bit, OD) are used by specific IODs (e.g. Parametric Map, some quantitative outputs) where sub-integer precision matters and Modality LUT semantics differ — Rescale Slope/Intercept may still apply, but there is no "Bits Stored masking" step because the values are natively floating point.

Common implementation mistake: writing a single generic "read pixel data as N-bit int" code path that silently misinterprets Float/Double Float Pixel Data as integer bytes, producing catastrophic numeric garbage (this is a hard failure, not a subtle one, but it is a real source of crashes/corrupted renders when a codebase adds support for a new IOD without updating the pixel decoder dispatch).

Extended Offset Table (7FE0,0001) / Extended Offset Table Lengths (7FE0,0002)

What it means: for encapsulated multi-frame Pixel Data with a large number of frames or frames exceeding legacy Basic Offset Table addressing limits, the Extended Offset Table provides 64-bit frame offsets and lengths, avoiding the 32-bit offset ceiling and item-length ambiguities of the classic Basic Offset Table embedded in the first fragment.

Common implementation mistake: only implementing the legacy Basic Offset Table path and failing on very large multi-frame encapsulated objects (common in whole-slide-adjacent or long cine capture use cases) where Extended Offset Table is present instead.

2.2 A determination walkthrough

L3 Given an arbitrary dataset, an engineer determines pixel structure in this order: (1) read Transfer Syntax UID to know if Pixel Data is native or encapsulated and which codec applies if encapsulated; (2) read Samples per Pixel, Photometric Interpretation, Rows, Columns, Bits Allocated, Bits Stored, High Bit, Pixel Representation, and, if Samples per Pixel > 1, Planar Configuration; (3) read Number of Frames if present and determine per-frame addressing (native: fixed stride; encapsulated: offset table or fragment boundaries); (4) only now decode raw bytes into a sample array; (5) proceed to Modality/VOI/Presentation. Steps 2 and 3 are metadata-only and must complete, without exception, before a single byte of Pixel Data is interpreted.

COMMON MISTAKE

Caching a "pixel reader" configured from the first frame or first instance of a series and reusing it across the whole series without re-validating Bits Stored / Photometric Interpretation / Planar Configuration per instance. Series are conventionally homogeneous but are not guaranteed to be, and Enhanced Multi-frame objects can vary parameters per frame via functional groups even within a single instance.

3 Raw Pixel Data at the Bit and Byte Level

This chapter treats Bits Allocated, Bits Stored, High Bit, and Pixel Representation as one coordinated system, with real hexadecimal bytes converted to real pixel values, for both signed and unsigned data.

3.1 The four-attribute system

L1 Native (non-encapsulated) Pixel Data is a flat array of fixed-width containers — 8-bit or 16-bit almost universally, occasionally requiring special packed handling below 8 bits which is rare in modern practice and not covered further here. Inside each container, only the low `Bits Stored` bits (measured from bit 0, the least significant bit) are meaningful; any bits above that, up to `Bits Allocated`, are unused padding that a conformant writer sets to zero but a robust reader must not trust blindly. `High Bit` tells you the bit position of the most significant meaningful bit; for DICOM's native encoding this is always `Bits Stored - 1`, which means `High Bit` is formally redundant with `Bits Stored` for native Pixel Data — but it must still be read and validated, both because some non-conformant data disagrees and because it is not redundant for certain other encodings.

Worked Example 3.1 — Unsigned 12-in-16 CT sample

Metadata: `Bits Allocated = 16`, `Bits Stored = 12`, `High Bit = 11`, `Pixel Representation = 0` (unsigned), little endian Transfer Syntax.

Two consecutive bytes read from the file for one pixel, in file order (little endian byte order):

Byte offset	+0 (low byte)	+1 (high byte)
Hex	0xB0	0x04
Binary	1011 0000	0000 0100

Assemble the 16-bit little-endian word: high byte becomes the upper 8 bits, low byte the lower 8 bits → 0000 0100 1011 0000 = 0x04B0 = 1200 decimal.

Now apply the `Bits Stored` mask. With `Bits Stored = 12`, only bits 0–11 are meaningful; mask with 0x0FFF:

```
0x04B0 & 0x0FFF = 0x04B0 → 1200 (unchanged, because bits 12-15 already happened to be zero here)
```

The stored pixel value is **1200**. This example was chosen deliberately so the mask is a no-op the first time — Example 3.2 shows why the mask is not optional in general.

Worked Example 3.2 — Same geometry, but with dirty high bits

Same metadata as 3.1. Bytes: 0xB0 0x74 → little-endian word = 0x74B0 = 0111 0100 1011 0000.

A reader that naively treats this as a plain unsigned 16-bit value reports **29872**. That is wrong: bits 12–15 (0111) are outside `Bits Stored` and must be discarded.

```
0x74B0 & 0x0FFF = 0x04B0 → 1200
```

The correct stored value is again **1200**. This is why "`Bits Allocated = Bits Stored`" is an unsafe assumption even when it is usually true in practice for a given scanner: a single unmasked read silently produces a value 24× too large, which will not window sanely and can be mistaken for a genuinely different tissue density if it slips past a HU sanity check.

3.2 Signed pixel representation and sign extension

L2 When `Pixel Representation = 1`, the meaningful bits (bit 0 through `High Bit`) are a two's-complement signed integer of width `Bits Stored`, right-justified in the container. To recover the correct signed value from the container: (1) mask to `Bits Stored` bits as above, (2) test whether the sign bit (bit position `Bits Stored - 1`) is set,

(3) if set, sign-extend by OR-ing in all bits above Bits Stored, i.e., subtract $2^{(\text{Bits Stored})}$ from the masked unsigned value.

Worked Example 3.3 — Signed 12-in-16 sample

Metadata: Bits Allocated = 16, Bits Stored = 12, High Bit = 11, Pixel Representation = 1 (signed), little endian.

Bytes: 0x9E 0x0F → little-endian word = 0x0F9E = 0000 1111 1001 1110.

Mask to 12 bits: 0x0F9E & 0x0FFF = 0x0F9E = binary 1111 1001 1110 (12 bits) = 3998 as an unsigned 12-bit reading.

Sign bit is bit 11 (the top bit of the 12-bit field), which is **1** here — this is a negative number. Sign-extend by subtracting $2^{12} = 4096$:

signed value = 3998 - 4096 = **-98**

A reader that forgets sign extension and reports the unsigned interpretation would output 3998 instead of -98 — a catastrophic error for any modality where negative stored values are legitimate (e.g. some MR reconstructions, subtraction-family images, or signed CT storage variants), because the Modality LUT will then be applied to a value in the wrong region of the intercept/slope mapping entirely.

3.3 Endianness, padding, and even-length rules

L3 Native Pixel Data byte ordering follows the Transfer Syntax: Explicit VR Little Endian and Implicit VR Little Endian both use little-endian sample byte order (least significant byte first); big-endian Transfer Syntax existed historically (Explicit VR Big Endian) but is retired in the current standard and should be treated as legacy-only. DICOM requires all data element values, Pixel Data included, to have even length; for a native Pixel Data element with an odd total byte count (which cannot actually happen for 8-bit-allocated data with an odd Rows×Columns×Samples-per-pixel product until you account for this rule), a single padding byte is appended and must be excluded from pixel interpretation, not treated as a stray extra sample.

Encapsulated (compressed) Pixel Data

When Transfer Syntax indicates a compressed codec (JPEG Baseline, JPEG 2000, JPEG-LS, RLE Lossless, HTJ2K, etc.), Pixel Data is encapsulated: a sequence of fragments delimited by DICOM Item tags (FFFE,E000), addressed via a Basic Offset Table (first item, may be empty) or an Extended Offset Table, with one or more fragments per frame. Bits Allocated/Stored/High Bit/Pixel Representation still describe the *decoded* sample format the codec must reproduce — they do not describe the compressed byte stream itself, which is why a decoder must decompress a full frame via the appropriate codec before Bits Stored masking and sign extension are even applicable operations. A frequent implementation trap is applying Bits Stored masking logic to compressed bytes directly (meaningless) instead of to the decompressed output samples.

Multi-frame organization: planar versus interleaved, native versus encapsulated

Layout	Native Pixel Data	Encapsulated Pixel Data
Single frame, single sample	Rows×Columns contiguous samples	One or more fragments → one decoded frame
Single frame, multi-sample (RGB)	Interleaved or planar per Planar Configuration	Codec-native (e.g. JPEG is always interleaved internally regardless of DICOM Planar Configuration, which for encapsulated Transfer Syntaxes is effectively fixed by the codec)
Multi-frame	Frames concatenated, frame stride = Rows×Columns×SamplesPerPixel×bytesPerSample	Frame N addressed via offset table or by sequential fragment consumption per frame boundary rules of the codec

Mistake: assuming Planar Configuration applies to encapsulated Transfer Syntaxes

For most compressed Transfer Syntaxes the codec's own internal color layout governs, and DICOM Planar Configuration for the element is conventionally documented as 0 (interleaved) or simply not the determining factor for interpreting decoded output — the decoded output layout is whatever the codec library hands back, and it is the decoder wrapper's job to normalize this to the layout the rest of the pipeline expects, not to look up Planar Configuration and hope the codec obeys it.

4 Photometric Interpretation

Photometric Interpretation is not cosmetic metadata. It is the single attribute that decides whether a stored sample represents a monochrome intensity with a stated polarity, an index into an arbitrary color palette, or a true color component — and it decides which downstream pipeline branch is legal to apply at all.

4.1 Stored sample interpretation versus display color interpretation

L1 A "stored sample interpretation" is the question "what does this integer mean as recorded" — e.g. "this is a linear grayscale intensity where larger is brighter" (MONOCHROME2) or "this is an index into a palette table, and I cannot know its color without consulting that table" (PALETTE COLOR). A "display color interpretation" is the RGB triplet or luminance value that actually reaches the screen after all pipeline stages. Photometric Interpretation only fixes the former; the latter still requires the full pipeline. Confusing these two is the single most common source of "my image looks inverted / discolored / washed out" bugs in early DICOM viewer implementations.

4.2 MONOCHROME1 and MONOCHROME2

L2 Both describe single-sample grayscale data across the full dynamic range implied by Bits Stored, differing only in polarity of the identity/default display mapping:

- **MONOCHROME2**: minimum stored value maps toward black, maximum stored value maps toward white (after Modality/VOI/Presentation, in the absence of any inverting Presentation LUT). This is the overwhelmingly common convention (CT, MR, most secondary captures).
- **MONOCHROME1**: minimum stored value maps toward white, maximum stored value maps toward black — used by some mammography and a subset of other modalities/vendors for specific display conventions.

Worked Example 4.1 — Same VOI output, opposite polarity

Assume an 8-bit display range [0,255] has already been produced by VOI transformation, with an IDENTITY Presentation LUT Shape, for both a MONOCHROME1 and a MONOCHROME2 image with identical underlying tissue contrast:

VOI output value	MONOCHROME2 displayed intensity	MONOCHROME1 displayed intensity
0	0 (black)	255 (white)
64	64 (dark gray)	191 (light gray)
128	128 (mid gray)	128 (mid gray, unchanged — the fixed point of the inversion)
255	255 (white)	0 (black)

The correct implementation is: $\text{displayed} = \text{MONOCHROME1} ? (\text{max_range} - \text{voi_output}) : \text{voi_output}$, applied as (or folded into) the Presentation LUT stage — not as an ad hoc special case bolted onto the VOI stage, because Presentation LUT Shape (INVERSE) and Photometric Interpretation (MONOCHROME1) are two independent, potentially compounding sources of polarity inversion that must both be tracked and, if both present, correctly combine to a net transformation rather than cancel out by coincidence of one being implemented and the other ignored.

4.3 PALETTE COLOR

L1 PALETTE COLOR has Samples per Pixel = 1: each stored sample is an *index*, not an intensity and not a color. The actual color for that index is looked up in three parallel LUTs (Red/Green/Blue Palette Color Lookup Table) carried in the dataset (Chapter 10). Two images with numerically identical stored samples but different palette tables can display as completely different colors — the stored sample alone carries no color information whatsoever.

4.4 RGB

L1 RGB has Samples per Pixel = 3: each pixel position stores three independent samples that are directly the red, green, and blue components (subject to Planar Configuration for their memory layout, Chapter 11). Unlike PALETTE COLOR, no lookup is required or permitted to determine displayed color from stored samples.

4.5 YBR-family interpretations, briefly

L2 For engineering completeness: YBR_FULL, YBR_FULL_422, YBR_PARTIAL_420, YBR_RCT, and YBR_ICT store luma/chroma-encoded samples (a color-space transform related to, but not bit-identical to, video YCbCr conventions), typically used with JPEG-family compressed Transfer Syntaxes where chroma subsampling and/or the JPEG-native color transform make YBR storage natural. A conformant decoder must apply the correct YBR→RGB inverse transform (with correct rounding, and correct handling of full-range vs. video-range chroma scaling depending on the specific YBR variant) before the data can be treated as RGB for any downstream purpose. This conversion is frequently handled transparently by the JPEG/codec library itself, but a viewer must still know which output format the library produced and must never re-apply a YBR→RGB transform to data the codec already converted, which would corrupt colors a second time.

4.6 Numerical example: stored value to displayed intensity

Worked Example 4.2 — Full chain for a single stored value under each polarity

Stored value = 3000 (Bits Stored = 12, unsigned). Rescale Slope = 1, Rescale Intercept = -1024 → Modality Value (HU) = $3000 - 1024 = 1976$ HU (dense bone/metal range). Window Center = 400, Window Width = 1500 (a "bone window"). Applying the linear VOI formula from Chapter 7 clips this to the top of the window, producing VOI output = 255 (8-bit output range) regardless of polarity, because 1976 HU is above the window's upper bound of 1150 HU.

Under MONOCHROME2 with IDENTITY Presentation LUT: displayed intensity = 255 (white). Under MONOCHROME1 with IDENTITY Presentation LUT: displayed intensity = 0 (black). The *clinical* value (1976 HU, dense material) is unchanged by polarity — only its visual encoding changes. This is exactly why Photometric Interpretation must never be treated as a "just flip it if it looks weird" cosmetic afterthought: getting it wrong doesn't just invert the picture, it silently swaps which visual extreme (black vs. white) a radiologist associates with a specific physical finding, on a system where that association is a learned clinical convention.

5 Modality LUT: Rescale and Explicit LUTs

The Modality LUT converts a stored sample — a number with meaning only relative to one acquisition's raw detector encoding — into a modality value with an actual physical or clinically standardized meaning, most famously the Hounsfield Unit in CT.

5.1 Why raw stored values are not clinically meaningful

L1 A CT scanner's reconstruction produces attenuation data that is conventionally stored as an integer offset from a fixed baseline to keep values non-negative and to fit compactly in an unsigned container, rather than storing signed Hounsfield Units directly. The Modality LUT undoes this storage convention and recovers the physically meaningful value. Two CT images from different scanner generations, or different reconstruction kernels, can use different Rescale Slope/Intercept pairs while representing the same clinical quantity (HU) once transformed — which is exactly the point: the transformation, not the raw number, is what's portable.

5.2 Rescale Slope / Rescale Intercept

Modality Value = Stored Value × Rescale Slope + Rescale Intercept
(DICOM tags: Rescale Slope (0028,1053), Rescale Intercept (0028,1052))

Worked Example 5.1 — Canonical CT case

```
Stored Value = 1200
Rescale Slope = 1
Rescale Intercept = -1024

HU = 1200 × 1 - 1024 = 176 HU
```

176 HU is in the soft-tissue-to-early-calcification range. The raw stored value 1200 has no clinical meaning on its own — it is an artifact of how this particular scanner chose to bias its output into an unsigned 12-bit container. A viewer that displayed "1200" as a pixel probe readout instead of "176 HU" would mislead a radiologist checking, for example, whether a renal cyst is simple (near 0–20 HU) or has protein/blood content (higher HU) — the raw number is roughly 6× larger and in a completely different part of the clinical scale.

Worked Example 5.2 — Negative modality values

```
Stored Value = 800 (unsigned, Bits Stored = 12)
Rescale Slope = 1
Rescale Intercept = -1024

HU = 800 - 1024 = -224 HU
```

–224 HU falls in fat tissue range. Note the Modality Value here is negative even though the stored value is a non-negative unsigned integer — this is entirely expected and is the mechanism by which an unsigned container encodes the full HU range from roughly –1024 (air) up through positive values for bone/metal. A downstream stage that assumes Modality Values are always ≥ 0 because Pixel Representation was 0 (unsigned) is conflating stored-value signedness with modality-value signedness, which are unrelated after the intercept is applied.

Worked Example 5.3 — Fractional / non-unit slope, precision

```
Stored Value = 512 (some quantitative modality with sub-unit scaling)
Rescale Slope = 0.0125
Rescale Intercept = 0.0

Modality Value = 512 × 0.0125 = 6.4 (physical units, e.g. SUV-adjacent or other quantitative scale)
```

Two engineering consequences: first, Modality Value is not necessarily integral, so any code path that stores it in an integer type silently truncates or rounds — this must be an explicit, documented rounding decision, not an implicit language cast. Second, Rescale Slope/Intercept in DICOM are encoded as Decimal String (DS) VR; parsing them with a fixed-precision or locale-sensitive float parser (e.g. one that expects comma decimal separators) is a real-world source of silent data corruption when a system is deployed across locales.

5.3 Explicit Modality LUT Sequence

L2 When Rescale Slope/Intercept cannot represent the required mapping (i.e., the modality-to-physical relationship is non-linear, or is linear but the vendor prefers explicit tabulation), the Modality LUT Sequence (0028,3000) carries an explicit lookup table with the same descriptor/data structure used by VOI and Palette Color LUTs. A dataset must not carry both Rescale Slope/Intercept and Modality LUT Sequence as competing definitions for the same purpose; per PS3.3, when Modality LUT Sequence is present it takes precedence and Rescale Slope/Intercept, if also present, should be ignored for the purpose of Modality transformation (implementations should treat this as a "prefer explicit LUT" rule and log/flag the unusual case of both being present with inconsistent implied behavior). The mechanics of indexing into this table are identical to the general LUT lookup covered in Chapter 6, applied here with the Modality LUT's own descriptor.

COMMON MISTAKE

Implementing only the Rescale Slope/Intercept path and silently falling back to "identity" (or crashing) when Modality LUT Sequence is present instead. This is a real interoperability gap: several PET, some nuclear medicine, and certain vendor-specific SOP Classes use explicit non-linear Modality LUTs, and treating stored values as HU-equivalent without any transformation in that case is a straightforward measurement error, not a display nuance.

5.4 Clinical significance: measurement domain versus display domain

L3 This is one of the most important distinctions in the entire manual, and it is worth stating plainly and repeating: **the Modality Value is the measurement domain** — it is what a radiologist places a region-of-interest tool on to read out "this lesion averages 45 HU," what a dose or SUV calculation is built from, what a CAD algorithm's thresholds are defined against. **Everything after the Modality LUT is the display domain** — window/level choices, inversion, GSDF calibration — and none of it should ever feed back into a measurement. A viewer that computes a "mean HU" measurement from windowed/VOI-transformed pixel values instead of from Modality Values is not producing an approximation of the correct answer; it is producing values that are a function of the current window/level setting the user happens to have open, which is clinically nonsensical for a supposedly quantitative measurement, and is a defect class real products have shipped with.

Modality / Context	Typical Modality LUT mechanism	Resulting physical unit
CT	Rescale Slope/Intercept, slope typically 1	Hounsfield Units (HU)
PET	Rescale Slope/Intercept and/or additional quantitative attributes (e.g. for SUV, computed from Radiopharmaceutical/Patient attributes, not Modality LUT alone)	Activity concentration; SUV is a further application-level computation beyond the Modality LUT itself
MR	Often identity or vendor-specific scaling; MR signal intensity is generally not an absolute physical unit across vendors/sequences	Arbitrary/relative signal intensity (context-dependent, not standardized like HU)
CR/DR (X-ray)	Rescale Slope/Intercept or explicit LUT mapping raw detector response toward a standardized exposure-related value	Detector/exposure-related value, modality- and vendor-defined

6 How a Non-Linear LUT Actually Works

This mechanism — descriptor, first mapped value, indexing, clamping — is identical across Modality LUT Sequence, VOI LUT Sequence, and Palette Color LUTs. Learn it once here; the later chapters only vary which numbers go in and what the output means.

6.1 The analogy, then the real mechanism

L1 A LUT is a lookup table exactly in the everyday sense: a phone book where you don't compute someone's number from their name, you look up the name and read the number sitting next to it. A DICOM LUT is the same idea specialized to integers: input sample value comes in, it's converted to a table row index, and the value sitting in the table at that row is the output — no formula, just tabulated correspondence, which is precisely what makes non-linear mappings representable without inventing a closed-form equation for each one.

6.2 LUT Descriptor and LUT Data

L2 Every DICOM LUT (Modality LUT Sequence's Item, VOI LUT Sequence's Item, and Red/Green/Blue Palette Color LUT) is carried as a three-value **LUT Descriptor** plus a **LUT Data** array:

LUT Descriptor field	Meaning
Value 1 — Number of Entries	How many entries LUT Data contains (0 is a special case meaning 65536 entries, a legacy encoding quirk you must handle explicitly)
Value 2 — First Input Value Mapped	The input sample value that corresponds to LUT Data entry index 0 (for Modality/VOI LUTs this is typically encoded as unsigned or signed per Pixel Representation context; for Palette Color LUTs it is always unsigned)
Value 3 — Bits per Entry	Bit depth of each output value stored in LUT Data (commonly 8 or 16)

An optional **LUT Explanation** (free text, human-readable, e.g. "SOFT TISSUE") may accompany the numeric definition purely for UI display purposes and must never be parsed for semantic meaning.

6.3 The lookup operation, worked from nothing

Worked Example 6.1 — Minimal non-linear LUT

Input domain and table:

```
Input: 0 1 2 3 4 5
LUT:  10 10 12 18 30 50
```

Suppose First Input Value Mapped = 0, Number of Entries = 6. The rule is simply:

```
index = input_value - First_Input_Value_Mapped
output = LUT_Data[index]
```

```
input 0 → index 0 → LUT[0] → 10
input 1 → index 1 → LUT[1] → 10
input 2 → index 2 → LUT[2] → 12
input 3 → index 3 → LUT[3] → 18
input 4 → index 4 → LUT[4] → 30
input 5 → index 5 → LUT[5] → 50
```

Worked Example 6.2 — Non-zero First Input Value Mapped

First Input Value Mapped = 100, Number of Entries = 6, same LUT Data as above.

```
Input = 103
index = 103 - 100 = 3
output = LUT_Data[3] = 18
```

Every input below 100, and every input at or above 106 (100 + 6), falls outside the table's defined domain and must be clamped, not indexed — indexing with a negative or out-of-bounds computed index is undefined and, in a systems language, a real memory-safety bug, not merely a logic bug.

Out-of-domain behavior

Condition	Required behavior
Input < First Input Value Mapped	Clamp: output = LUT_Data[0] (the first table entry)
Input > First Input Value Mapped + Number of Entries - 1	Clamp: output = LUT_Data[last] (the final table entry)
Input exactly at First Input Value Mapped	output = LUT_Data[0], the boundary case, must not be off-by-one
Input exactly at the last mapped value	output = LUT_Data[Number_of_Entries - 1]

L3 Note there is no interpolation between adjacent DICOM LUT entries defined by the standard's basic LUT lookup mechanism — the table is dense over its declared domain (one entry per representable input value in that range), so "interpolation" in the classic image-processing sense is not part of the base LUT lookup operation itself. Where interpolation legitimately enters the pipeline is in *linear* VOI windowing (a closed-form formula, Chapter 7) and in optional smoothing some viewers apply when down-sampling a fine LUT for display purposes — that is an application-level rendering choice, not a DICOM LUT semantic.

6.4 Applying this identically to Modality, VOI, and Palette Color LUTs

The mechanism above is reused without modification in three places, differing only in what the input and output represent:

LUT type	Input domain	Output meaning
Modality LUT Sequence	Stored Value	Modality Value (e.g. non-linear physical unit)
VOI LUT Sequence	Modality Value (post-Modality-LUT)	VOI-transformed / display-intensity-domain value
Palette Color LUT (Red/Green/Blue, each separately)	Stored Value (PALETTE COLOR index)	One color channel's intensity

Implement the lookup exactly once, as a single reusable function taking (Number of Entries, First Input Value Mapped, Bits per Entry, LUT Data, input) → output, and call it from all three sites. A codebase with three independently-written, subtly-different LUT lookup implementations is a common source of the class of bug where Modality LUT indexing works but Palette Color indexing off-by-ones at the domain boundary, or vice versa.

7 VOI LUT and Windowing

VOI (Value of Interest) transformation compresses a clinically meaningful numeric range — often far wider than a monitor's usable contrast steps — into a bounded display-intensity range, selecting which part of that range gets the available contrast. It is a display transformation. It must never alter the underlying Modality Value or Pixel Data.

7.1 Why VOI exists

L1 A chest CT's Modality Values (HU) span roughly -1000 (air) to $+3000$ or more (dense bone/metal) — a dynamic range of several thousand distinct clinically meaningful steps. A clinical display, even a high-end grayscale medical monitor, presents on the order of 8-11 usable bits of distinguishable perceptual steps. Displaying the full HU range linearly onto that limited output would compress lung parenchyma, mediastinum, and bone into a handful of indistinguishable gray levels each. Windowing selects a sub-range (the "window") and stretches only that sub-range across the full available display contrast, at the cost of clipping everything outside it to pure black or pure white.

7.2 Linear VOI: Window Center and Window Width

L2 DICOM PS3.3 C.11.2.1.2 defines the linear windowing function precisely; it is not simply "clip and rescale" in an arbitrary way — the boundary behavior matters and is exactly specified. Using Window Center (WC) and Window Width (WW), and defining the output range as $[y_{\min}, y_{\max}]$ (e.g. 0 to 255 for 8-bit display output):

```
lower bound = WC - WW/2
upper bound = WC + WW/2

if x ≤ WC - (WW-1)/2 - 0.5 → output = y_min
if x > WC - (WW-1)/2 + (WW-1) → output = y_max
otherwise → output = ((x - (WC-0.5)) / (WW-1) + 0.5) × (y_max-y_min) + y_min
```

The $-0.5 / WW-1$ terms are not stylistic; they are the standard's exact boundary-centering definition (this is the "DICOM linear" form, chosen so that the window is centered on WC with well-defined behavior for both even and odd WW, and so that a WW of 1 is a legal, meaningful degenerate case representing a hard binary threshold). Reimplementing this from a simplified "textbook" formula such as $(x-l_o)/(hi-l_o)$ without the half-pixel offsets produces output that is subtly shifted from the DICOM-conformant result — usually invisible on casual inspection but measurable, and inconsistent with how the same image would render in a reference viewer.

Worked Example 7.1 — Abdominal soft tissue window

Window Center = 40, Window Width = 400, output range [0, 255].

Input HU	Region	Output
-200	below lower bound ($40 - 200 = -160$)	0 (clipped black)
0	inside window	$\approx 128 \times \text{scaled position}$ — specifically $((0-39.5)/399 + 0.5) \times 255 \approx 104.6 \rightarrow 105$
40	window center	$((40-39.5)/399 + 0.5) \times 255 \approx 128.3 \rightarrow 128$
100	inside window	$((100-39.5)/399 + 0.5) \times 255 \approx 166.9 \rightarrow 167$
240	at/near upper bound ($40+200=240$)	≈ 254 -255 (near-clip)
1000	far above upper bound	255 (clipped white)

Note that HU 1000 (dense bone) and HU 240 (near the window edge) both saturate toward white under this soft-tissue window — this is the entire point: bone is intentionally sacrificed to contrast resolution here so that the 0-100 HU soft-tissue range gets the display's full dynamic range. A "bone window" (e.g. WC 400, WW 1500) would instead compress this same soft-tissue range into a narrow near-uniform gray band while spreading contrast across the bone range.

7.3 Non-linear VOI LUT Sequence

L2 When a linear window cannot represent the desired mapping (common for vendor-tuned "soft" display curves, mammography presets, or perceptually-tuned mappings), VOI LUT Sequence (0028,3010) carries an explicit table using the exact lookup mechanism from Chapter 6, with input domain = Modality Value and output = the display-intensity-domain value.

Worked Example 7.2 — Non-linear VOI lookup

```
First Input Value Mapped = -50
Number of Entries = 8
LUT Data = [0, 10, 40, 90, 150, 200, 230, 255]

Modality Value = 0
index = 0 - (-50) = 50 — out of range! (only indices 0..7 valid)
```

This flags a modeling error in a naive worked case: with only 8 entries starting at -50, the domain only covers -50..-43. A real non-linear VOI LUT for CT-scale data would declare Number of Entries in the thousands (matching the Modality Value dynamic range it's meant to cover, e.g. 4096 entries for a 12-bit-derived range) — the lookup mechanics are identical to Example 6.2, just at production scale. The critical implementation point illustrated here is that Number of Entries must genuinely span the expected input range, and an engineer validating a vendor's VOI LUT Sequence should explicitly check domain coverage against the image's actual Modality Value range rather than assuming it.

7.4 Multiple VOI alternatives and preset selection

L3 Window Center, Window Width, and VOI LUT Sequence are all multi-valued attributes — a single image can carry several window presets (e.g. "lung," "bone," "mediastinum" for CT chest) or several VOI LUT Sequence items (e.g. multiple vendor display curves), each optionally labeled by the corresponding Window Center & Width Explanation (0028,1055) or LUT Explanation. DICOM does not mandate which one a viewer must display first; it is standard practice, and a reasonable default, to present the first listed value/item initially and expose the rest as user-selectable alternatives, with the choice persisted as needed (e.g. in a Presentation State). A viewer must not

silently average or blend multiple presets, and must not assume Window Center/Width and VOI LUT Sequence are both present and interchangeable — per PS3.3, if VOI LUT Sequence is present it takes precedence over Window Center/Width for the purposes of the default VOI transformation.

COMMON MISTAKE

Treating the presence of multiple Window Center/Width values as an error condition or arbitrarily taking a numeric average of them. Multiple values represent distinct, purposeful presets (e.g. different tissue windows) — averaging them produces a window that is clinically meaningless and matches none of the vendor's intended presentations.

7.5 Why window/level must never mutate the underlying pixel data

The VOI stage is, by definition, reversible in the sense that changing WC/WW and re-rendering must reproduce a correct image from the same unmodified Modality Values every time. An architecture that "bakes" a chosen window into the stored Pixel Data (common in naive implementations that render once to an 8-bit bitmap and discard the original data) destroys the ability to re-window, destroys any downstream quantitative measurement, and is not what DICOM Presentation States are for — a Presentation State records the *chosen* VOI parameters alongside a reference to the unmodified image, precisely so the transformation stays reversible and auditable (Chapter 13).

8 Presentation LUT and the Grayscale Standard Display Function

Modality LUT converts stored values into physical meaning. VOI LUT selects and stretches a range of interest into display-intensity space. Presentation LUT is the final, distinct step that maps that display-intensity value into a standardized, polarity-correct P-Value ready for perceptually-linearized display — and it is not the same operation as either of the first two, even though all three are "just LUTs."

8.1 Why a third LUT stage is needed at all

L1 VOI output is, conceptually, "how bright should this be, on some display-oriented scale" — but it does not yet account for whether the image is MONOCHROME1 (needing inversion) or whether the presentation context calls for an inverted display independent of photometric polarity (e.g. mammography soft-copy reading preferences), and it does not yet account for the fact that raw digital driving levels sent to a monitor do not produce a perceptually linear luminance response. Presentation LUT handles the first concern (polarity / P-Value mapping); GSDF (Chapter 8.4) handles the second (perceptual linearization). Bundling all of this into "the window/level code" is exactly the kind of stage-collapsing this manual has been warning against since Chapter 1.

8.2 Presentation LUT Shape and Presentation LUT Sequence

L2 Presentation LUT Shape (2050,0020) takes the enumerated value `IDENTITY` (pass VOI output through unchanged into P-Value space) or `INVERSE` (invert). Alternatively, Presentation LUT Sequence (2050,0010) carries an explicit non-linear table using the same Chapter 6 mechanism, input = VOI output, output = P-Value.

Worked Example 8.1 — Identity vs. inverse		
VOI output: 0 1 2 3 4 (8-bit) 0 64 128 192 255		
VOI output	IDENTITY P-Value	INVERSE P-Value (255-x)
0	0	255
64	64	191
128	128	127
192	192	63
255	255	0

8.3 Distinguishing Modality, VOI, and Presentation LUT explicitly

	Modality LUT	VOI LUT	Presentation LUT
Input	Stored Value	Modality Value	VOI output value
Output	Modality Value (physical/clinical unit)	Display-intensity-domain value	P-Value (presentation-ready)
Purpose	Recover physical meaning	Select/compress dynamic range for display	Polarity correction and perceptual-mapping-ready output
Alters clinical measurement?	Defines it	No — display-only	No — display-only
Typical controlling attributes	Rescale Slope/Intercept, Modality LUT Sequence	Window Center/Width, VOI LUT Sequence	Presentation LUT Shape, Presentation LUT Sequence

L3 A common shortcut in simple viewers is to fold Presentation LUT into the VOI stage by just choosing to invert the window's output range for MONOCHROME1 images and calling it done. This works only as long as INVERSE Presentation LUT Shape and explicit Presentation LUT Sequence are never encountered independently of Photometric Interpretation — which is not a safe assumption once Presentation States (Grayscale Softcopy Presentation State, Chapter 13) enter the picture, because a Presentation State can specify INVERSE display independent of the underlying image's own MONOCHROME1/2 polarity, and the two inversions must compose correctly (two inversions cancel; one applied twice by mistake also cancels, silently producing a non-inverted image where an inverted one was intended — a real, subtle bug class).

8.4 Grayscale Standard Display Function (GSDF, PS3.14)

L1 Human perception of brightness is not linear in physical luminance — equal steps in luminance are not perceived as equal steps in brightness, especially at low luminance. DICOM PS3.14 defines the GSDF: a standardized curve relating a perceptually uniform "Just-Noticeable Difference" (JND) index to target luminance, so that a display calibrated to GSDF produces perceptually equal steps for equal steps in P-Value (or in the DICOM-specified 10-bit-plus driving level derived from it), regardless of the specific monitor's native gamma or luminance response.

L2 Conceptually: P-Value $\rightarrow$ JND index (via the DICOM-specified GSDF mapping) $\rightarrow$ target luminance (via the monitor's own calibration curve, often via a per-monitor lookup table applied in the graphics pipeline or in monitor firmware) $\rightarrow$ actual photons. The exact GSDF formula (a piecewise polynomial-in-log-luminance relation defined in PS3.14 Section B) is not reproduced here verbatim because it is a display-calibration standard best consulted directly for implementation, but the engineering point to internalize is architectural, not formulaic.

CRITICAL SEPARATION: PIXEL-VALUE TRANSFORMATION VS. MONITOR LUMINANCE CALIBRATION

GSDF conformance is fundamentally a property of the display chain (monitor calibration, graphics card LUT, or a dedicated calibration LUT applied just before scan-out), not something every DICOM viewer must reimplement as a per-pixel software LUT baked into the image processing pipeline. Many production viewers rely on OS-level or monitor-level GSDF calibration (medical-grade monitors typically self-calibrate to GSDF in hardware/firmware) and simply need to output correctly-computed P-Values; some viewers additionally apply a software GSDF-aware calibration LUT when running on non-calibrated consumer displays to approximate perceptual linearity. Do not conflate "my viewer computes correct P-Values" with "my viewer is GSDF-conformant end to end" — the latter also depends on the display hardware/software stage that the DICOM pixel pipeline itself does not control.

8.5 Why a viewer developer should still understand GSDF even if the monitor "handles it"

L3 Three practical reasons: (1) on non-calibrated consumer hardware — a real and common deployment target for non-primary-diagnostic viewing, mobile apps, referring-physician portals — the application is the only layer that can apply any perceptual correction at all, and doing so requires understanding what "correct" means; (2) debugging a "washed out" or "low contrast" report from the field requires knowing whether the defect is in P-Value computation (your bug) or in display calibration (a monitor/environment issue, not your bug) — conflating the two wastes engineering time chasing the wrong layer; (3) any inline soft-copy annotation or contrast-sensitive UI element (e.g. rendering a measurement overlay at a luminance intended to be perceptually consistent with the underlying image) needs the same perceptual model the image itself is calibrated against.

9 Masking and Subtraction

Digital Subtraction Angiography and related temporal-subtraction techniques rely on removing a static "mask" frame from a "live" (contrast-filled) frame so that only the change — contrast agent filling a vessel — remains visible. Getting the pipeline position of this step wrong corrupts contrast in a way that can hide or fabricate apparent vascular filling.

9.1 What a mask image is

L1 In X-Ray Angiographic imaging, a "mask" frame is typically acquired before contrast agent injection (showing only background anatomy — bone, soft tissue, no contrast-filled vessels), and one or more "live" frames are acquired during and after injection (showing the same anatomy plus contrast-filled vessels). Subtracting mask from live cancels the static anatomy, leaving primarily the vessel filling — the diagnostically relevant signal — with dramatically improved conspicuity compared to viewing the live frame alone.

9.2 Where subtraction sits in the pipeline

L2 Subtraction operates on **Modality Values**, after the Modality LUT and before VOI (see Figure 1.1). This ordering matters: subtracting raw stored values would be incorrect whenever the mask and live frames could have different Rescale Slope/Intercept (uncommon within a single series but not disallowed in general), and subtracting after VOI would mean subtracting already-clipped, already-nonlinearly-compressed display values, which destroys the linear relationship needed for the subtraction to isolate contrast agent signal correctly.

Worked Example 9.1 — Basic subtraction

```
Mask pixel (Modality Value) = 120
Live pixel (Modality Value) = 170

Difference = 170 - 120 = 50
```

This positive difference of 50 represents the contrast-agent-attributable signal at this pixel. The difference image is then itself subject to a VOI transformation of its own — typically a window centered near zero with a width chosen to make small vessel-filling differences visible — distinct from the window that would be applied to the live frame alone.

9.3 Signed differences, absolute difference, and polarity

L2 The raw difference (live — mask) is signed: contrast filling typically increases attenuation (positive difference in this convention), but motion, registration error, or non-contrast physiological change can produce negative differences at some pixels. Two display conventions exist:

- **Signed subtraction with polarity-aware VOI:** keep the signed difference and window it so zero maps to mid-gray, positive differences brighten or darken consistently by convention, and negative differences do the opposite — this preserves directionality of change, useful for distinguishing genuine contrast filling from misregistration artifact (which often produces sharp positive/negative edge pairs at high-contrast boundaries).
- **Absolute-difference (magnitude) subtraction:** take $|\text{difference}|$ and window from zero upward — simpler, but discards the sign information that can help a reader distinguish true vessel filling from a misregistration edge artifact.

DICOM does not mandate one convention over the other at the Pixel Data level for a subtraction-family SOP Class; the choice is governed by the specific IOD's presentation intent and, where applicable, the accompanying Presentation State. An implementation must be explicit and consistent about which convention it uses and must document it, because the same underlying subtraction data displayed under the two conventions is not visually interchangeable.

9.4 Registration, motion artifact, and temporal misregistration

L3 Subtraction assumes mask and live frames are spatially registered — the patient (and the anatomy of interest) must not have moved between mask and live acquisition. Any motion (patient movement, breathing, bowel peristalsis, cardiac motion, table movement) produces registration error that manifests as high-contrast "double edge" artifacts at anatomical boundaries in the subtracted image, which can be mistaken for or can obscure genuine vascular pathology. Some workflows apply pixel shift (a manual or semi-automated small translational correction applied to the mask before subtraction) or more general non-rigid registration to compensate; when present, this is an application/workflow-level correction applied to the mask frame before the subtraction arithmetic, not a DICOM pixel-transformation-pipeline stage in itself, and it must be applied consistently and be user-auditable (i.e., a viewer should indicate when pixel shift or registration correction is active, since it changes what the displayed subtraction image represents relative to the raw acquired frames).

9.5 Multi-frame organization for subtraction data

Mask and live frames are typically carried within the same Multi-frame Pixel Data element (for classic X-Ray Angiographic SOP Classes) with frame relationship attributes (e.g. Mask Subtraction Sequence, Recommended Viewing Mode, Mask Frame Numbers) indicating which frame(s) serve as mask for which live frame(s); Enhanced Multi-frame IODs express equivalent relationships via functional groups. An implementation must read these relationship attributes explicitly rather than assuming frame 0 is always the mask and all subsequent frames are live — conventions vary, and some acquisitions use a single mask against many live frames (typical DSA run) while others may define frame-specific mask relationships.

9.6 Clinical applications and risk of incorrect implementation

Application	Role of subtraction
Digital Subtraction Angiography (DSA)	Primary technique — isolates contrast-filled vasculature from static anatomy for vascular intervention guidance and diagnosis.
Temporal subtraction (e.g. chest radiography)	Subtracts a prior exam from a current exam (after registration) to highlight interval change — a distinct clinical use of the same subtraction concept, generally implemented at the application/workflow level rather than as an intrinsic Pixel Data relationship within one acquisition.
Bone-suppression-style processing	Conceptually related dual-energy or trained-model subtraction techniques to suppress bony structures and improve soft-tissue conspicuity; typically a vendor-specific post-processing algorithm rather than a raw mask/live DICOM subtraction, but shares the same "isolate the signal of interest by removing a baseline" principle.

L3 Implementing subtraction at the wrong pipeline stage, with the wrong sign convention undocumented, or without correct mask-frame identification, produces an image that still looks plausibly like "a subtraction image" — grayscale, vessel-like structures visible — while being quantitatively and potentially diagnostically wrong: contrast filling can appear attenuated, artifactually enhanced, or spatially displaced relative to the true acquisition. Because the failure mode is visually plausible rather than obviously broken, subtraction logic warrants the same rigorous numerical validation against a trusted reference implementation described in Chapter 19, not visual-inspection-only testing.

10 Palette Color

PALETTE COLOR exists because some acquisition or processing pipelines (ultrasound flow overlays, certain nuclear medicine and PET colorized outputs, some legacy secondary captures) produce a single indexed channel meant to be rendered through an arbitrary, often non-grayscale, color mapping — and encoding this as an index-plus-table is dramatically more compact and more faithful to the intended display than converting to RGB at acquisition time.

10.1 Why Palette Color exists, and how it differs fundamentally from RGB

L1 A PALETTE COLOR stored sample carries no color information by itself — it is purely an index into three separate lookup tables, one per channel. This is fundamentally different from RGB, where each stored sample is directly a color-channel intensity. Two PALETTE COLOR datasets with identical Pixel Data but different palette tables render as different colors entirely; this can never happen with RGB, where the stored samples are the color.

10.2 The relevant attributes

Attribute	Tag	Meaning
Red Palette Color LUT Descriptor	(0028,1101)	Number of Entries, First Input Value Mapped, Bits per Entry — for the red channel table
Green Palette Color LUT Descriptor	(0028,1102)	Same, green channel
Blue Palette Color LUT Descriptor	(0028,1103)	Same, blue channel
Red/Green/Blue Palette Color LUT Data	(0028,1201)/(1202)/(1203)	The actual per-channel table values

L3 Per PS3.3, the three descriptors must share the same Number of Entries and First Input Value Mapped (they describe three parallel tables indexed identically by the same stored sample value) — only Bits per Entry can, in principle, differ, though in practice it is normally consistent across the three as well. A defensive implementation should validate this invariant and treat a mismatch as a non-conformant dataset warning rather than silently indexing each channel table independently with potentially different domains, which would misalign the three channels for a given input.

10.3 Worked example

Worked Example 10.1 — One stored value becoming an RGB triplet

```
Pixel (stored index) = 25

Red LUT[25] = 200
Green LUT[25] = 100
Blue LUT[25] = 50

Result = RGB(200, 100, 50)
```

Each channel LUT is looked up using the exact mechanism of Chapter 6 — $\text{index} = \text{stored_value} - \text{First_Input_Value_Mapped}$, then $\text{output} = \text{LUT_Data}[\text{index}]$, independently for R, G, and B, with the results assembled into a single RGB triplet per pixel. If Bits per Entry = 16 while the target display buffer is conventionally 8-bit-per-channel, the 16-bit LUT output must be scaled down (typically by taking the high byte, or by proper rounding division by 257 to map [0,65535] onto [0,255] exactly at both endpoints) rather than truncated by a naive right-shift, which introduces a small but real banding/rounding bias.

10.4 Bit depth, endianness, and signedness of LUT entries

L2 Palette Color LUT Data entries are conventionally unsigned (there is no equivalent of Pixel Representation for palette LUT entries — color intensities are inherently non-negative). Entries are packed at Bits per Entry (commonly 8 or 16) following the Transfer Syntax's byte order for multi-byte entries. A subtlety worth flagging explicitly: even when Bits Allocated for the underlying Pixel Data is 8 (a single-byte index), the LUT Data entries themselves are frequently 16 bits per entry regardless — Bits per Entry in the LUT Descriptor is independent of the Pixel Data's own Bits Allocated, and conflating the two is a real parsing mistake.

10.5 Why Palette Color is fundamentally different from RGB — restated as an implementation rule

COMMON MISTAKE

Treating a PALETTE COLOR image's stored samples as if they were already grayscale intensities (displaying the raw index values directly) or, conversely, as if Samples per Pixel were already 3 and reading the byte stream as interleaved RGB. Both produce a plausible-looking but entirely wrong image; the correct handling is always: read Photometric Interpretation, branch explicitly on PALETTE COLOR, and route through the three-channel LUT expansion before any RGB-oriented code (windowing, compositing, GPU texture upload as RGB8) is allowed to touch the data.

11 RGB Pixel Data

RGB images (Samples per Pixel = 3, Photometric Interpretation = RGB) are the simplest photometric case conceptually, but the most common place implementations silently corrupt color by mishandling Planar Configuration.

11.1 Interleaved vs. planar memory layout

L1 Planar Configuration = 0 means samples are interleaved per pixel — the natural order most consumer image formats use. Planar Configuration = 1 means each component is stored as a complete separate plane, all red samples for the whole image first, then all green, then all blue.

Worked Example 11.1 — Byte layout for a 2×2 RGB image

Assume pixels (in row-major order) with colors $P0=(R0,G0,B0)$, $P1=(R1,G1,B1)$, $P2=(R2,G2,B2)$, $P3=(R3,G3,B3)$, 8 bits per sample.

Planar Configuration = 0 (interleaved):

```
R0 G0 B0 R1 G1 B1 R2 G2 B2 R3 G3 B3
```

Planar Configuration = 1 (planar):

```
R0 R1 R2 R3 G0 G1 G2 G3 B0 B1 B2 B3
```

A decoder that hardcodes the interleaved layout and receives Planar Configuration = 1 data will read $R0$ as red, $R1$ as green, $R2$ as blue for what it believes is pixel 0 — producing a scrambled, banded, or falsely-colored image, not a subtly wrong one, but the failure can still look like "a real image, just weird colors" rather than obvious garbage, which is what makes it easy to ship undetected in casual QA.

11.2 Reconstructing pixels from either layout

```
// Planar Configuration = 0
R = bytes[3*i + 0]; G = bytes[3*i + 1]; B = bytes[3*i + 2];

// Planar Configuration = 1, N = Rows*Columns
R = bytes[i]; G = bytes[N + i]; B = bytes[2*N + i];
```

L3 For multi-frame RGB, the plane vs. interleave organization repeats within each frame (frame stride = Rows × Columns × 3 × bytesPerSample regardless of Planar Configuration) — Planar Configuration does not change how frames are concatenated, only how the three components are arranged within one frame's byte range.

11.3 Bits Allocated / Bits Stored for RGB components

Each of the three samples independently follows the same Bits Allocated/Bits Stored/High Bit/Pixel Representation rules from Chapter 3. 8-bit-per-channel RGB (Bits Allocated = Bits Stored = 8) is overwhelmingly the common case for true-color DICOM secondary captures and photographic modalities; higher-bit-depth RGB exists but is rarer and must not be assumed absent without checking the metadata.

11.4 Why RGB does not need Modality/VOI processing the way MONOCHROME does

L3 RGB stored samples are already intended as direct display-ready color intensities (subject only to color-management concerns below) — there is no physical-measurement domain analogous to Hounsfield Units to recover via a Modality LUT, and no clinically meaningful "windowing" of a color photograph in the

way CT contrast windowing works. Consequently, Modality LUT Module and VOI LUT Module attributes are not expected to be meaningfully present/applied for RGB Photometric Interpretation, and a pipeline implementation should treat RGB as branching around those stages entirely (see the lower branch in Figure 1.1) rather than attempting to apply an identity Modality/VOI transform to each channel out of a desire for code-path uniformity — applying even a no-op stage in the wrong place for RGB is a sign the branch logic itself is not correctly separated from the grayscale path.

11.5 When color management still matters after decoding

Even though RGB skips Modality/VOI, decoded RGB samples are not automatically in a well-defined, cross-device-consistent color space by DICOM's core Image Pixel Module alone. Where color fidelity matters (dermatology photography, pathology, endoscopy), an ICC Profile (via the ICC Profile attribute, 0028,2000, and related Color Space attributes) may be present and should be applied through a color management transform to a device-independent connection space before final display — a separate concern from, and layered on top of, the base RGB decode described in this chapter, and outside DICOM's grayscale-oriented Modality/VOI/Presentation LUT model entirely.

11.6 Multi-frame RGB and common Planar Configuration errors

Mistake: assuming Planar Configuration is constant across a series or session

Planar Configuration is a per-instance (and, for compressed Transfer Syntaxes, effectively codec-determined per the discussion in Chapter 3.3) attribute. A pipeline that reads it once from the first image of a study and reuses that value for subsequent instances can misinterpret any instance that legitimately differs, including instances produced by different acquisition or processing software within the same study.

12 Spatial Transformations

Flips, rotations, and orientation labeling are not interchangeable operations on a pixel array — one is a display convenience, the other is a statement about anatomical left/right, and confusing them is the single class of bug in this entire manual with the most direct potential for a wrong-side clinical error.

12.1 Pixel-array transformation versus patient-space transformation

L1 A **pixel-array transformation** (flip horizontal, flip vertical, rotate 90°) rearranges which stored sample appears at which screen position, with no reference to the patient at all — it is purely a display-convenience operation, the kind a user might apply because they prefer viewing a chest film "as if facing the patient." A **patient-space transformation** is the mapping between pixel array coordinates and real, physical, patient-relative 3D coordinates (Image Orientation (Patient), Image Position (Patient), Pixel Spacing) — it is a statement of anatomical fact derived from the acquisition geometry, not a display preference, and it is what determines whether "left" on the screen actually corresponds to the patient's left or right side, and whether a measurement tool reports millimeters correctly.

12.2 Image Orientation (Patient) and Image Position (Patient)

L2 Image Orientation (Patient) (0020,0037) is a 6-value direction-cosine pair: the first three values are the direction cosines of the first row of pixels (i.e., the direction you move in patient space as Column index increases), the second three are the direction cosines of the first column of pixels (i.e., the direction you move in patient space as Row index increases), both expressed in the DICOM Patient Coordinate System (x increasing to patient left, y increasing to patient posterior, z increasing to patient superior, per PS3.3 C.7.6.2.1.1). Image Position (Patient) (0020,0032) gives the x,y,z patient-space coordinate of the center of the first (top-left) voxel of the image (Row 0, Column 0).

```
Patient(row, col) = ImagePositionPatient
+ col × PixelSpacing[1] × RowDirectionCosines
+ row × PixelSpacing[0] × ColumnDirectionCosines
```

(Pixel Spacing is [row spacing, column spacing] — the physical distance between adjacent rows and between adjacent columns respectively — and must be paired correctly with which direction-cosine triplet it scales; swapping this pairing produces a spatially valid-looking but metrically distorted mapping.)

Figure: image coordinates to patient coordinates



Figure 12.1. The image-to-patient coordinate mapping is a well-defined affine transform driven entirely by Image Orientation (Patient), Image Position (Patient), and Pixel Spacing — it is independent of any display-convenience flip/rotate a viewer applies to the on-screen pixel array.

12.3 Patient orientation and multi-frame/slice geometry

L3 Patient Orientation (0020,0020) and Patient Position (0018,5100, e.g. HFS = Head First Supine, FFS = Feet First Supine, HFP, FFP) provide additional, coarser orientation context used primarily for whole-study/whole-series conventions and for sanity-checking the finer per-image direction cosines. For a multi-slice or multi-frame series, slice ordering and inter-slice spacing must be derived consistently — conventionally by projecting each slice's Image Position (Patient) onto the shared slice-normal direction (the cross product of the row and column direction cosines) and sorting by that scalar projection, rather than trusting instance/frame number alone, since instance numbering is not guaranteed to correspond to spatial order in every acquisition or transfer scenario.

12.4 Display-convenience flips/rotations versus orientation correctness

A viewer is free to apply pixel-array flips/rotations for on-screen presentation convenience (and to record such choices in a Presentation State's Spatial Transformation attributes, Chapter 13), but any such transformation must be tracked and composed correctly with the underlying patient-space mapping so that: (1) any orientation marker or laterality annotation drawn on screen still correctly reflects true anatomical left/right after the display transform, and (2) any measurement tool still reports correct physical distances/areas by consulting Pixel Spacing and the patient-space geometry, not the possibly-flipped screen-pixel geometry directly.

CLINICAL SAFETY: L/R ERRORS

An incorrect spatial transformation — most dangerously, an unaccounted-for horizontal flip applied between the true patient-space mapping and the displayed orientation marker — can cause a viewer to correctly display anatomy while incorrectly labeling or implying the wrong side, precisely the class of error implicated in real wrong-site clinical incidents involving imaging misread. Any code path that applies a flip/rotation for display must recompute or invert the accompanying orientation label/marker in the same operation, never independently, and this logic deserves dedicated automated tests (Chapter 19), not visual spot-checks alone.

13 Shutters, Overlays, Annotations, and Presentation States

Everything in this chapter sits architecturally downstream of, and logically separate from, the pixel-value pipeline of Chapters 5–9. Confusing "what the pixel value is" with "what is drawn on top of or masked around the pixel" is a recurring source of both rendering bugs and, for burned-in versus separate annotation, real privacy and clinical-record issues.

13.1 Shutters

L1 A shutter defines a region of the displayed image that should be excluded from diagnostic display — typically because it corresponds to detector area outside the actual X-ray beam collimation, which would otherwise show as raw, clinically meaningless (and sometimes distractingly bright or noisy) detector signal at the image periphery.

L2 Shutter Shape (0018,1600) may be RECTANGULAR, CIRCULAR, POLYGONAL, or BITMAP, each with its own geometry-defining attributes (e.g. Shutter Left/Right Vertical Edge and Shutter Upper/Lower Horizontal Edge for rectangular; Center and Radius for circular; a Vertices list for polygonal; an actual Overlay-plane-style bitmap for BITMAP). Shutter Presentation Value (0018,1622) specifies the P-Value (not a raw stored pixel value) to display for pixels outside the shutter region — conventionally a fixed black or a specific mid-gray, applied as a compositing step, not as a modification of the underlying stored Pixel Data.

L3 Architecturally, the shutter is a geometric masking operation applied at composition time (typically right after or alongside the Presentation LUT stage, per Figure 1.1/1.2's "Shutter Transformation" position), not a pixel-value transformation in the Modality/VOI/Presentation sense — it does not change any in-shutter pixel's value at all, and its determination of inside/outside is a pure geometry test (point-in-rectangle, point-in-circle, point-in-polygon, or bitmap lookup) evaluated per output pixel position. Incorrectly applying the shutter before VOI transformation, for example, would mean the excluded region's Shutter Presentation Value gets re-windowed as if it were real Modality Value data — clearly wrong, since Shutter Presentation Value is defined directly in P-Value space, downstream of VOI.

13.2 Overlays, graphic annotations, and burned-in annotations — three distinct things

Mechanism	What it is	Lives in Pixel Data?
Overlay Data (60xx,3000, legacy Overlay Plane Module)	A separate 1-bit-per-pixel bitmap layer (e.g. graphics, ROI outlines, some ultrasound annotation) carried as its own data element, composited over the image at display time	No — a distinct element, never mixed into Pixel Data
Graphic/Text Annotations (via a Presentation State's Graphic Annotation Sequence)	Vector graphics/text (lines, points, measurements, labels) defined in image or display coordinates, rendered by the viewer at composition time	No — referenced separately, rendered on demand
Burned-in annotation	Text/graphics that have been rasterized directly into the Pixel Data itself at acquisition or export time (common on some ultrasound and secondary-capture images, and an important de-identification concern)	Yes — physically part of the pixel samples, cannot be toggled off or separately controlled

L3 This distinction must never be treated as an implementation detail. Overlay Data and Presentation State annotations are separable, toggleable, and — critically for de-identification workflows — do not require pixel-level scrubbing to remove patient-identifying text, because the text simply isn't in the pixel samples. Burned-in annotation, indicated by the Burned In Annotation attribute (0028,0301) when present, requires actual pixel-level redaction (blackening the affected region of the pixel array, or re-acquisition/re-export without burn-in) for any de-identification workflow — a metadata-only de-identification pass that scrubs DICOM tags but leaves burned-in pixel text intact is a real, documented failure mode in clinical data-sharing pipelines and a genuine privacy exposure, not a cosmetic gap.

13.3 Displayed Area and Presentation Size Mode

Displayed Area attributes (Displayed Area Top Left Hand Corner / Bottom Right Hand Corner, 0070,0052/0070,0053, plus Presentation Size Mode, 0070,0100) within a Presentation State describe which sub-region of the image should be shown and at what sizing convention (e.g. TRUE SIZE for physically-calibrated 1:1 display, MAGNIFY, SCALE TO FIT). These attributes describe cropping/zooming intent, distinct from both the pixel-value pipeline and from Image Orientation/Position, and must be composed on top of both without altering either.

13.4 DICOM Presentation States: the architectural role

L1 A Grayscale Softcopy Presentation State (GSPS) — and its color/pseudo-color/blending-family relatives — is a separate DICOM object that references one or more images and records a complete, reproducible description of how those images should be displayed: VOI parameters, Presentation LUT, spatial transform (rotation/flip), Displayed Area, shutters, and graphic/text annotations. Conceptually:

Image Object + Presentation State = Intended Visualization

L3 This separation exists specifically so that the underlying image (the measurement) and a particular way of looking at it (the presentation) can be independently stored, exchanged, and reasoned about — a radiologist's exact window/level, annotations, and measurements from a diagnostic read can be saved, retrieved, and reproduced exactly by any conformant viewer without ever touching or duplicating the original Pixel Data. This underpins diagnostic reproducibility (a report can be re-verified against the exact view the radiologist saw), hanging-protocol-driven review workflows, and safe multi-user collaboration on the same underlying image without one user's display preference silently overwriting another's, which would be the inevitable outcome of any architecture that "bakes" display choices into the image itself.

End-to-end example

Worked Example 13.1 — Image plus Presentation State

A CT image with Window Center/Width = 40/400 embedded as its own default. A radiologist reviewing for a bone query changes the window to 400/1500, rotates the image 90° for a preferred reading orientation, adds a text annotation "possible cortical irregularity" with an arrow, and defines a circular shutter to de-emphasize an irrelevant table edge artifact. All of this is captured in a new GSPS object referencing the original image's SOP Instance UID. The original image's Pixel Data, Window Center/Width, and all Image Pixel Module attributes remain completely unmodified. A colleague opening the same image later, without loading the GSPS, sees the original default 40/400 soft-tissue window and no annotation; loading the GSPS reproduces the radiologist's exact bone-window, rotated, annotated view.

14 Cornerstone, OHIF, and Framework Architecture

If modern web-based imaging applications already build on frameworks like Cornerstone (a rendering/toolkit library) and OHIF (a full viewer application built on Cornerstone), why does an engineer still need everything in this manual? Because frameworks implement mechanism, and mechanism without correct semantics still produces wrong images.

14.1 The layered architecture, conceptually

```
DICOM instance(s)
↓
DICOM parser / decoder (dataset + Transfer Syntax handling)
↓
Image loader (resolves an addressable "image ID" to decoded pixel data + metadata)
↓
Pixel representation (typed array of decoded samples, per Chapters 2-3)
↓
Modality / VOI / color transformations (Chapters 5-10)
↓
Rendering engine (compositing, spatial transform, annotation overlay)
↓
WebGL / Canvas / GPU
↓
Display
```

L1 This is a general architecture description of how modern browser-based DICOM viewer stacks are typically organized, not a claim about any single library's exact internal module boundaries, which evolve across versions. The point of drawing it this way is that every stage in this manual maps onto a real layer in a real production architecture — none of it is academic.

14.2 What frameworks typically provide, and what remains application/domain responsibility

Concern	Typically framework-provided	Typically still your responsibility
DICOM parsing, Transfer Syntax decoding	Yes, for common/well-supported Transfer Syntaxes and codecs	Handling of unsupported/vendor-specific Transfer Syntaxes, malformed data, and graceful degradation
Image loaders, metadata providers	Yes, general-purpose loader/provider patterns	Wiring your specific data source (PACS, WADO-RS, local files) and ensuring metadata provided matches what the loaded pixel data actually represents
Modality LUT, VOI (window/level) application	Yes, standard linear windowing and common LUT application	Verifying correctness for non-standard/vendor LUT usage, and validating output against a trusted reference for your specific clinical use cases
Color conversion (RGB, YBR, Palette Color)	Commonly supported for standard cases	Confirming coverage for the specific photometric interpretations and codecs your data actually contains; not every viewer supports every photometric interpretation out of the box
GPU/WebGL rendering pipeline	Yes	Performance tuning for your specific data volumes (Chapter 18), and correctness of any custom shader-based processing you add
Multi-frame / cine handling	Commonly supported	Correct frame-relationship interpretation for specialized IODs (e.g. subtraction mask relationships, Enhanced Multi-frame functional groups)
Derived images (e.g. MPR, fusion)	Toolkit support commonly available	Clinical correctness of the derivation algorithm and its interaction with the base pixel pipeline
Presentation State support	Partial to full, framework/version dependent	Validating exactly which Presentation State features (shutters, specific annotation types, Presentation LUT Sequence) are actually implemented versus silently ignored by your specific version
Application-specific clinical workflow behavior	Not provided	Entirely your responsibility — hanging protocols, reporting integration, institution-specific defaults

14.3 Why understanding the standard still matters when using a framework

L3 Six concrete, recurring reasons, none of which are hypothetical:

- **Incorrect metadata still produces incorrect output.** A framework applying a mathematically correct VOI transformation to a Modality Value that was itself computed from mis-parsed Bits Stored/Pixel Representation still yields a wrong image — the framework has no way to know your upstream metadata handling is broken.
- **Unsupported SOP Classes or transformations require application logic.** Not every framework version supports every IOD (e.g. certain Enhanced Multi-frame variants, certain Presentation State graphic types, certain segmentation objects) out of the box; recognizing an unsupported case, rather than silently mis-rendering it, requires knowing what correct support would look like.
- **Clinical workflows require customization** that necessarily sits above the generic rendering library — institution-specific hanging protocols, specialty-specific default windows, integration with reporting/RIS — and customizing correctly requires understanding what the underlying pipeline stage you're customizing actually does.
- **Performance optimization requires understanding the pipeline** (Chapter 18) — deciding what can be precomputed, cached, or moved to a shader requires knowing which stages are linear/composable and which are not (Chapter 1.2).
- **Debugging requires knowing what correct looks like.** When a customer reports "the image looks wrong," distinguishing a framework bug, a metadata bug, a data-source bug, and a genuinely correct-but-unexpected rendering (e.g. a legitimately MONOCHROME1 image) requires the diagnostic vocabulary this manual builds.

- **Interoperability problems require understanding DICOM, not assuming the library is authoritative.** When your viewer disagrees with a vendor's modality console or another PACS viewer on the same image, the standard — not whichever viewer you're most familiar with — is the arbiter of which one is conformant.

DISCIPLINE FOR THIS CHAPTER

This manual does not make unsupported claims about specific library internals, because those internals change across versions and are best verified against the library's own current documentation and source for any given release. The durable content here is the layered architecture and the division-of-responsibility reasoning, which remains valid regardless of which specific framework version you are auditing.

15 Clinical Safety: Why Pixel Transformation Is a Safety Concern

Every failure mode below follows the same three-step pattern: a technical mistake, an incorrect displayed image, and a plausible clinical consequence. None of these are sensationalized — they follow directly and mechanically from the mechanisms explained in Chapters 2–13.

Technical mistake	Incorrect displayed image	Possible clinical consequence
Treating MONOCHROME1 as MONOCHROME2 (ignoring polarity)	Full tonal inversion — dense structures appear dark instead of light and vice versa	A reader accustomed to the standard convention may misjudge tissue density/character at a glance, particularly under time pressure; inverted mammography-family images are a specifically documented risk area
Applying Window Center/Width directly to raw stored values, skipping Modality LUT	For CT with Rescale Intercept -1024 , the entire window is shifted by 1024 HU-equivalent units — a "soft tissue" window (WC 40) effectively becomes centered near HU -984 , i.e., displaying almost entirely air/lung-density content as mid-gray while genuine soft tissue saturates to white	Soft tissue contrast is effectively destroyed; subtle masses or fluid collections that depend on 0–100 HU contrast resolution become indistinguishable
Assuming Bits Allocated equals Bits Stored (no masking of unused high bits)	Occasional pixels with garbage high bits read as extreme outlier values, corrupting auto-windowing heuristics or producing sporadic bright/dark speckle	Auto-window algorithms driven by actual min/max can produce a badly mis-scaled default window; sporadic artifactual bright pixels can mimic or mask small high-contrast findings (e.g. calcifications, metal fragments)
Missing or incorrect sign extension for signed Pixel Representation	Negative Modality Values wrap to large positive stored-value-equivalents before rescale, producing wildly wrong Modality Values for affected pixels	Quantitative measurements (HU, signal intensity) on affected pixels are simply wrong, with no visual indication distinguishing them from valid data
Ignoring Planar Configuration for RGB	Scrambled or falsely-colored image, sometimes resembling three ghosted/misregistered copies of the same structure in different colors	Color-dependent findings (e.g. vascularity on color Doppler overlay, tissue coloration in endoscopy/dermatology capture) become uninterpretable or misleading
Treating PALETTE COLOR as RGB (or vice versa)	Either a false-color scramble (palette-as-RGB) or a flat grayscale-looking render that discards the intended color mapping (RGB-as-palette, or the reverse)	Functional/flow overlays (e.g. Doppler flow color coding conveying direction/velocity) lose their encoded meaning entirely
Incorrect spatial orientation / unaccounted display flip	Anatomically correct-looking image with an incorrect or unreconciled laterality marker	Wrong-side clinical error risk — among the most serious failure modes in this entire table (Chapter 12.4)
Incorrect Pixel Spacing interpretation (e.g. swapped row/column spacing, or Pixel Aspect Ratio ignored)	Anatomy appears stretched/compressed along one axis	Distance and area measurements (e.g. lesion size, stent sizing) are systematically wrong
Incorrect shutter handling (e.g. applying shutter value before VOI, or ignoring shutter)	Out-of-collimation detector noise displayed as if diagnostic, or conversely valid image content incorrectly blacked out	Distracting artifact can obscure peripheral findings; incorrect masking of real anatomy is a direct information loss

<i>geometry)</i>		
<i>Wrong frame ordering in multi-frame/cine or multi-slice series</i>	<i>Temporally or spatially scrambled sequence</i>	<i>Motion/perfusion/contrast-timing assessment becomes unreliable; 3D reconstruction (MPR) from mis-ordered slices produces geometrically wrong volumes</i>
<i>Incorrect subtraction (wrong pipeline stage, wrong mask identification)</i>	<i>Vessel contrast attenuated, artifactually enhanced, or spatially displaced</i>	<i>Vascular stenosis or occlusion could be under- or over-represented</i>
<i>Incorrect LUT indexing (off-by-one at domain boundary, missing clamp)</i>	<i>Localized banding, wraparound artifacts, or out-of-bounds crashes at specific value ranges</i>	<i>Specific density/intensity ranges systematically mis-displayed, potentially the exact range relevant to a particular pathology</i>
<i>Integer overflow / precision loss in Rescale or LUT arithmetic</i>	<i>Wrapped or clipped values at extremes of the dynamic range</i>	<i>Extreme-density findings (dense calcification, metal, air) misrepresented</i>
<i>Incorrect rounding in linear VOI computation</i>	<i>Small, usually sub-visually-perceptible shifts in boundary pixel intensities</i>	<i>Generally low risk in isolation, but can cause measurable disagreement with a reference viewer during validation/conformance testing, and can compound with other small errors</i>
<i>Incorrect clipping (e.g. no clamping at LUT/window boundaries)</i>	<i>Modulo-wrapped or otherwise nonsensical values for out-of-window data instead of clean black/white saturation</i>	<i>Extreme-density regions render as arbitrary mid-tones rather than being clearly flagged (by full saturation) as outside the current window, potentially confused with genuine intermediate-density tissue</i>

READING THIS TABLE CORRECTLY

None of these failure modes require malicious or grossly incompetent code — every one of them is the direct, mechanical result of skipping or reordering a single pipeline stage described earlier in this manual, in code that otherwise "runs" and "shows an image." This is precisely why pixel transformation correctness cannot be validated by visual inspection alone (Chapter 19) and why code review for imaging pipelines should explicitly check pipeline-stage ordering and boundary handling, not just algorithmic correctness of each stage in isolation.

16 End-to-End Worked Case Studies

Each case study below follows one or more raw pixel values through every pipeline stage, from encoded bytes to a final displayed value, using realistic metadata.

16.1 Case Study A — CT, MONOCHROME2

METADATA

Rows = 512, Columns = 512, Bits Allocated = 16, Bits Stored = 12, High Bit = 11, Pixel Representation = 0, Photometric Interpretation = MONOCHROME2, Rescale Slope = 1, Rescale Intercept = -1024, Window Center = 40, Window Width = 400, Presentation LUT Shape = IDENTITY.

Pixel A1 — soft tissue

```
Hex bytes (little endian): 0x38 0x04 → word 0x0438 = 1080
Mask to 12 bits: 0x0438 & 0x0FFF = 0x0438 = 1080 (Stored Value)
Modality: HU = 1080 × 1 - 1024 = 56 HU
VOI (WC 40, WW 400): output = ((56 - 39.5)/399 + 0.5) × 255 ≈ 147
Presentation (IDENTITY): P-Value = 147
Display (MONOCHROME2, no inversion): mid-light gray, consistent with soft tissue
near the window center
```

Pixel A2 — cortical bone

```
Hex bytes: 0xE8 0x09 → word 0x09E8 = 2536, mask 0x0FFF → 2536 (Stored Value)
HU = 2536 - 1024 = 1512 HU
VOI (WC 40, WW 400): far above upper bound (40+200=240) → output = 255 (clipped
white)
P-Value = 255 → pure white on screen
```

The radiologist sees near-uniform white for this pixel — correct and expected for dense cortical bone under a soft-tissue window; a bone-window preset (e.g. WC 400/WW 1500) would be selected to differentiate cortical detail instead.

16.2 Case Study B — MONOCHROME1 (polarity reversal)

Same metadata as Case Study A except Photometric Interpretation = MONOCHROME1. Pixel A1's P-Value computation is unchanged (147) up through the Presentation LUT stage; MONOCHROME1's default polarity then inverts the final display mapping:

```
Displayed intensity (MONOCHROME1) = 255 - 147 = 108 (darker gray, versus 147 lighter
gray under MONOCHROME2)
```

The clinical quantity (56 HU) and even the P-Value (147) are identical to Case Study A — only the final visual polarity differs, exactly as established in Chapter 4.2.

16.3 Case Study C — Explicit Modality LUT

METADATA

Modality LUT Sequence present: Number of Entries = 4096, First Input Value Mapped = 0, Bits per Entry = 16, defining a non-linear stored-value-to-physical-value mapping (Rescale Slope/Intercept absent/superseded).

Pixel C1

```
Stored Value = 1080
index = 1080 - 0 = 1080
Modality Value = ModalityLUT_Data[1080] - e.g. 212 (a vendor-defined non-linear
physical value at this table entry)
...VOI and Presentation proceed exactly as in Case A, but operating on 212 rather
than on a slope/intercept-derived HU
```

The engineering point: once the Modality Value is obtained — whether via Rescale Slope/Intercept or via explicit LUT lookup — every downstream stage is identical code. The Modality LUT's internal mechanism should be fully encapsulated behind a single "get Modality Value for this Stored Value" function.

16.4 Case Study D — Non-linear VOI LUT

Pixel D1

```
Modality Value (HU) = 56
VOI LUT Sequence: First Input Value Mapped = -1024, Number of Entries = 4096, Bits
per Entry = 8
index = 56 - (-1024) = 1080
VOI output = VOILUT_Data[1080] - e.g. 162 (a vendor perceptually-tuned curve value,
not necessarily equal to the linear-window result of 147 computed in Case A for the
same HU)
```

Note the non-linear VOI curve can legitimately produce a different output than the linear WC/WW formula for the same Modality Value — both are valid VOI mechanisms; a viewer must apply whichever one the dataset specifies (VOI LUT Sequence takes precedence over Window Center/Width when both present, per Chapter 7.4) rather than defaulting to whichever is easier to implement.

16.5 Case Study E — PALETTE COLOR

Pixel E1

```
Stored Value (index) = 180
Red/Green/Blue Palette Color LUT Descriptor: Number of Entries = 256, First Input
Value Mapped = 0, Bits per Entry = 8 (all three channels)
index = 180 - 0 = 180
Red LUT[180] = 255, Green LUT[180] = 60, Blue LUT[180] = 20
Displayed RGB = (255, 60, 20) - a saturated red-orange, typical of a "high flow
velocity" color-Doppler-style palette entry
```

16.6 Case Study F — RGB with Planar Configuration

Pixel F1

```
Samples per Pixel = 3, Photometric Interpretation = RGB, Planar Configuration = 1,  
Rows×Columns = 4 (toy example), pixel index i = 2  
Byte stream: R0 R1 R2 R3 G0 G1 G2 G3 B0 B1 B2 B3 = 10 20 200 90 15 25 180 95 5 12  
150 80  
R = bytes[2] = 200, G = bytes[4+2] = 180, B = bytes[8+2] = 150  
Displayed RGB = (200, 180, 150) — a warm tan tone
```

A decoder hardcoded for Planar Configuration = 0 would instead read bytes[6],[7],[8] = (95, 15, 25) — a completely different, wrong color for the same logical pixel.

16.7 Case Study G — Multi-frame organization

A cardiac cine series: Number of Frames = 30, Rows = 512, Columns = 512, Samples per Pixel = 1, Bits Allocated = 8, native (uncompressed) Pixel Data. Frame stride = $512 \times 512 \times 1 \times 1$ byte = 262,144 bytes. Frame k (0-indexed) occupies byte range $[k \times 262144, (k+1) \times 262144)$. A viewer must read Number of Frames before attempting any frame-indexed access, and must recompute the stride correctly if Samples per Pixel or Bits Allocated differ from this toy example's assumptions.

16.8 Case Study H — DSA subtraction

Mask frame (frame 0) Modality Value at a given pixel = 118. Live frame (frame 12, during contrast injection) Modality Value at the same pixel = 246. Signed difference = $246 - 118 = 128$, a strong positive signal indicating substantial contrast filling at this location; this difference value is then windowed (e.g. centered near zero, width tuned to make small vessel differences visible) exactly as described in Chapter 9.2–9.3, entirely independent of whatever window the live frame alone would use for anatomical viewing.

16.9 Case Study I — Spatial orientation

Image Orientation (Patient) = [1,0,0, 0,1,0] (row direction = pure patient-x/left, column direction = pure patient-y/posterior — a standard axial slice orientation), Image Position (Patient) = [-150, -150, 45], Pixel Spacing = [0.7, 0.7]. The patient-space coordinate of pixel (row=100, col=200) is: $x = -150 + 200 \times 0.7 = -10$, $y = -150 + 100 \times 0.7 = -80$, $z = 45$ (unchanged within the slice) → patient coordinate (-10, -80, 45) mm. Any display-convenience flip applied on top of this must be tracked separately and must not alter this underlying computed geometry (Chapter 12.4).

16.10 Case Study J — Presentation State

Base image: MONOCHROME2 CT slice, default Window Center/Width 40/400. A GSPS references this image and specifies Window Center/Width 400/1500 (bone), Presentation LUT Shape IDENTITY, a 90° clockwise rotation, and one graphic annotation (an arrow with text "cortical step-off?") anchored at image coordinate (312, 210). Rendering this combination: load the base image's Pixel Data unmodified → apply Modality LUT (unchanged from the base image's own Rescale Slope/Intercept, since the Presentation State does not redefine Modality LUT) → apply the GSPS's VOI (400/1500) instead of the image's own default (40/400) → apply Presentation LUT (IDENTITY) → rotate the composed raster 90° clockwise → composite the annotation at a coordinate correctly transformed through that same rotation so it still points at the intended anatomical location rather than at wherever image coordinate (312,210) would have been pre-rotation.

17 Mistakes That Look Reasonable but Are Wrong

Each of these is a plausible-sounding shortcut an experienced, competent engineer might reach for under deadline pressure — not a beginner error. That is precisely why each deserves explicit attention in code review.

"I'll apply Window Center/Width directly to the raw stored CT values — it's just a linear formula anyway."

Why someone might make it: both the Modality LUT (for a typical CT with slope 1) and the linear VOI transform are affine, so it's tempting to believe the composition is "close enough" without explicitly applying the intercept.

Why it's wrong: Rescale Intercept for CT is essentially always non-zero (typically -1024); skipping it shifts the entire effective window by 1024 units, which is not a rounding error, it is a completely different tissue range (Chapter 15's second row).

Correct implementation: always compute Modality Value first, unconditionally, even when Rescale Slope = 1, before any VOI computation touches the value.

Clinical consequence: soft-tissue contrast destroyed; see Chapter 15.

"Bits Allocated tells me the sample width; I don't need to separately handle Bits Stored."

Why someone might make it: for 8-bit data the two are almost always equal, and many test images happen to have Bits Allocated = Bits Stored even at 16 bits, so the bug hides during casual testing.

Why it's wrong: 10- and 12-bit modalities are common and leave unused high bits that must be masked (Chapter 3.1).

Correct implementation: always mask to Bits Stored, unconditionally, as a matter of policy — it is a no-op when the two happen to be equal and a required correction when they don't.

Clinical consequence: corrupted auto-windowing and quantitative measurements; see Chapter 15.

"All grayscale DICOM images are MONOCHROME2; I'll hardcode that."

Why someone might make it: MONOCHROME2 is overwhelmingly the more common convention across most modalities an engineer is likely to test against first.

Why it's wrong: MONOCHROME1 is a real, standard, actively-used value, and treating it identically to MONOCHROME2 fully inverts the image (Chapter 4.2).

Correct implementation: branch explicitly on Photometric Interpretation for every image, every time; never assume based on modality alone.

Clinical consequence: full tonal inversion, misread density/character.

"PALETTE COLOR is basically color data; I'll just treat it as if Samples per Pixel were 3."

Why someone might make it: the end result is color, so it's tempting to route it through the same code path as RGB.

Why it's wrong: Samples per Pixel is 1 for PALETTE COLOR; stored samples are indices, not color components (Chapter 4.3, Chapter 10).

Correct implementation: branch on Photometric Interpretation before deciding sample count interpretation; expand through the three-channel LUT before any RGB-shaped code touches the data.

Clinical consequence: false-color scramble; loss of encoded functional information (e.g. Doppler flow direction/velocity).

"Planar Configuration only matters for exotic formats; I'll assume interleaved."

Why someone might make it: interleaved is the dominant convention in most non-DICOM image formats the engineer has prior experience with.

Why it's wrong: Planar Configuration = 1 is legal, standard, and encountered in real datasets (Chapter 11).

Correct implementation: always read and branch on Planar Configuration explicitly for Samples per Pixel > 1 native Pixel Data.

Clinical consequence: scrambled/false color, as in Chapter 15.

"Presentation LUT and VOI LUT are basically the same thing — both are 'apply a LUT to make it look right.'"

Why someone might make it: both use the identical Chapter 6 lookup mechanism and both sit in the "display-facing" part of the pipeline.

Why it's wrong: they operate on different input domains and solve different problems — VOI selects/compresses dynamic range, Presentation LUT handles polarity/perceptual-readiness (Chapter 8.3). Merging them ad hoc breaks correct composition with independent MONOCHROME1 and Presentation-State-level inversion.

Correct implementation: implement them as two distinct, separately-testable stages, even if they are later composed for performance (Chapter 1.2, Chapter 18).

Clinical consequence: silent double-inversion or missed inversion under combined MONOCHROME1 + Presentation-State scenarios.

"All LUTs are linear enough that I can just min/max normalize instead of doing a real table lookup."

Why someone might make it: a min/max linear stretch is simpler to implement and looks visually similar for many everyday images.

Why it's wrong: explicit non-linear LUTs (Modality, VOI, Palette Color) exist precisely because a linear mapping cannot represent vendor-tuned or physically-correct curves (Chapter 6, 7.3).

Correct implementation: implement the general LUT lookup mechanism once and always use it when an explicit LUT Sequence is present, reserving the linear formula strictly for the Window Center/Width case where the standard actually defines a closed-form linear transform.

Clinical consequence: loss of vendor-intended contrast tuning; disagreement with reference viewers.

"My viewer framework handles rendering, so clinical correctness is the framework's problem."

Why someone might make it: the framework does correctly implement the standard transformations it supports.

Why it's wrong: a framework cannot validate your metadata, cannot know about SOP Classes/features it doesn't support, and cannot make your application-specific customizations correct (Chapter 14.3).

Correct implementation: treat framework correctness as necessary, not sufficient; validate the full path from your specific data sources through your specific customizations.

Clinical consequence: any of the failure modes in Chapter 15, now with an added false sense of security.

"I'll just apply the window/level and save the result back into the pixel array so I don't have to recompute it every frame."

Why someone might make it: looks like a reasonable caching/performance optimization.

Why it's wrong: this destroys the original Modality Values, breaking re-windowing, quantitative measurement, and Presentation State reproducibility (Chapter 7.5, Chapter 13.4).

Correct implementation: cache computed display buffers separately from source Pixel Data; never overwrite the source.

Clinical consequence: permanent, silent loss of the ability to re-window or measure correctly; corrupted downstream Presentation States.

"I'll flip the image for display without worrying about the orientation marker — it's just a display preference."

Why someone might make it: the flip itself is a legitimate, standard display option.

Why it's wrong: pixel-array flips must be composed with patient-space orientation labeling, or laterality becomes wrong while the image still looks anatomically plausible (Chapter 12.4).

Correct implementation: any spatial transform applied for display must recompute the associated orientation label/marker in the same code path, never independently.

Clinical consequence: wrong-side error risk — the single most serious failure mode in this manual.

18 Engineering Architecture and Performance

Correctness first, then performance — and performance work in this domain is almost entirely about deciding what can be precomputed or moved to the GPU without silently changing clinical semantics.

18.1 Reference architecture

```
DICOM Reader
↓
Transfer Syntax Decoder
↓
Pixel Sample Decoder (Ch. 2-3: masking, sign extension, planar/interleaved)
↓
Photometric Interpreter (Ch. 4: branch grayscale / RGB / palette / YBR)
↓
Modality Transformation (Ch. 5)
↓
VOI Transformation (Ch. 7, after optional Mask Subtraction, Ch. 9)
↓
Presentation Transformation (Ch. 8)
↓
Color / Grayscale Conversion (Palette-RGB, YBR-RGB if not already done, Ch. 10-11)
↓
Spatial Transformation (Ch. 12)
↓
Overlay / Annotation Compositing (Ch. 13)
↓
Rendering Backend → GPU / Display
```

L3 Each arrow above should correspond to a real function boundary with a documented, unit-testable input/output contract. Semantic boundaries that must remain explicit even under aggressive optimization: Modality vs. VOI (different physical meaning, Chapter 8.3), VOI vs. Presentation (independent polarity sources must compose correctly, Chapter 8.3), and pixel-value transformation vs. everything in Chapter 13 (geometric/compositing operations, not value transformations).

18.2 What is safe to combine, and what is not

Combination	Safe?	Condition
Rescale Slope/Intercept + linear VOI window, pre-composed into one affine map	Yes	Both are linear/affine; recompute whenever either input changes; do not use when Mask Subtraction sits between them
Rescale Slope/Intercept + explicit non-linear VOI LUT, pre-composed into one combined LUT	Yes	Resample: for each possible Stored Value, compute Modality Value via slope/intercept, then look up VOI LUT at that Modality Value, store the composed result — do not attempt to algebraically merge a linear stage into a table stage other than by this resampling approach
VOI + Presentation LUT (IDENTITY)	Yes	IDENTITY is a no-op; safe to fold trivially
VOI + Presentation LUT (INVERSE or explicit Sequence)	Yes, with care	Compose as a single combined LUT exactly as above; must still be able to expose the pre-Presentation VOI value separately if MONOCHROME1 polarity needs independent handling (Chapter 8.3)
Modality LUT + Mask Subtraction + VOI, all pre-composed into one static LUT	No	Mask Subtraction depends on two distinct pixel positions (mask frame, live frame) at Modality Value level — it is not a per-pixel-independent LUT operation and cannot be folded into a single 1D lookup table
Spatial transform folded into the value-transformation LUT	No	Spatial transform is a geometric remapping of pixel position, categorically different from a value transformation; conflating the two breaks testability and reuse

18.3 CPU vs. GPU implementation strategies

L3 For a single static image, a precomputed combined Modality+VOI+Presentation LUT (Chapter 18.2) is small (at most a few thousand entries for typical Bits Stored ranges) and can be applied per-pixel on the CPU cheaply, or uploaded as a 1D lookup texture and applied in a fragment shader for GPU-accelerated windowing — the latter is the standard technique behind interactive real-time window/level dragging in browser-based viewers (WebGL/WebGPU), because recomputing and re-uploading only the small LUT texture per window/level change, rather than reprocessing the full image on the CPU per frame, is dramatically cheaper for large images or multi-frame cine.

- **LUT caching / precomputed LUTs:** cache the composed Modality+VOI+Presentation LUT keyed by (Rescale Slope, Rescale Intercept, WC, WW, Presentation LUT Shape) so that repeated renders at the same window/level (e.g. scrolling through a series at a fixed window) avoid LUT recomputation.
- **Shader-based windowing:** pass Modality Value (or even Stored Value plus slope/intercept as shader uniforms) into a fragment shader that performs the linear VOI formula directly per-pixel on the GPU, avoiding any CPU-side LUT texture entirely for the common linear-window case — simpler than LUT-texture approaches and well-suited to continuous window/level dragging.
- **Zero-copy pipelines / texture formats:** decode Stored Values directly into a GPU texture format that preserves full Bits Stored precision (e.g. a 16-bit integer or float texture) rather than lossy-downsampling to 8-bit before the VOI stage, which would discard precision the VOI transformation needs to correctly reproduce narrow windows (e.g. a lung window's fine gradations).
- **WebAssembly (WASM):** useful for CPU-side decode paths (e.g. custom or less-common codec support, or Modality/VOI computation for non-GPU-accelerated deployment targets) where near-native performance is needed within a browser sandbox without a GPU dependency.

18.4 Performance considerations by data volume

Scenario	Primary concern	Typical mitigation
Large CT series (hundreds of slices)	Memory footprint, decode throughput	Streamed/progressive loading, per-slice lazy decode, GPU texture streaming
4K/8K high-resolution images (e.g. digital pathology-adjacent, some high-res secondary capture)	Texture upload bandwidth, tiling	Tiled rendering, mipmap-style level-of-detail for zoomed-out views
Multi-frame cine (cardiac, angio)	Frame decode/display rate for smooth playback	Pre-decode a frame buffer ahead of playback position; shader-based windowing to avoid per-frame CPU recompute
Browser deployment generally	Main-thread blocking during decode	Web Workers for decode, transferable ArrayBuffers to avoid copy overhead into the render thread

THE GOVERNING RULE FOR ALL OPTIMIZATION IN THIS DOMAIN

Every optimization technique above is a restructuring of where and when a transformation is computed — never a change to what the transformation computes. If an optimization changes the numeric output for any input compared to the unoptimized reference implementation, it is a correctness bug wearing a performance-improvement disguise, and must be caught by the validation methodology in Chapter 19 before it ships.

19 Testing and Validation Methodology

Visual similarity is not proof of DICOM-semantic correctness. Two viewers can produce images that look alike to a casual observer while disagreeing on individual pixel values, on which pipeline stage introduced a given effect, or on both.

19.1 Why visual similarity alone is insufficient

L3 A visually acceptable image can result from two compensating errors (e.g. a missing Rescale Intercept combined with a coincidentally-adjusted default window that happens to look plausible on one test image but fails on the next), and a genuinely correct image can look unfamiliar to a tester used to a different reference viewer's default presentation choices (e.g. a different default window preset). Numerical, stage-by-stage comparison against a trusted reference implementation is the only reliable validation method for a medical-image rendering pipeline; visual inspection is a useful secondary sanity check, never the primary correctness signal.

19.2 Test case construction, by concern

Concern	Test construction approach	Expected result to verify
Signed pixels	Synthetic dataset with Pixel Representation = 1, known signed values including boundary values (most negative, most positive, zero, -1)	Correct sign-extended integer for each, exactly matching hand-computed expected values (Chapter 3.2)
Unsigned pixels, varying Bits Stored	Synthetic datasets at Bits Stored = 8, 10, 12, 16, each with deliberately "dirty" unused high bits set	Identical decoded value regardless of dirty-bit state, matching the masked expected value (Chapter 3.1)
MONOCHROME1 / MONOCHROME2	Identical Modality/VOI-stage values, differing only in Photometric Interpretation	Exact polarity-inverted relationship ($255-x$ for 8-bit) between the two outputs (Chapter 4.2, 16.2)
Explicit Modality LUT	Small, fully-specified LUT with known non-trivial values at several indices including both domain boundaries	Exact table value at each tested index; correct clamping outside domain (Chapter 6.3)
Rescale Slope/Intercept	Known stored value, known slope/intercept, including a fractional-slope case	Exact arithmetic result, with documented, tested rounding behavior if stored in an integer type (Chapter 5.2)
Linear VOI	Full sweep of Modality Values across, below, at, and above the window boundaries, including $WW = 1$ degenerate case	Exact match to the standard's boundary-centered linear formula (Chapter 7.2), not a naive simplified formula
Non-linear VOI	Explicit small VOI LUT with known values	Exact table lookup result, correct clamping (Chapter 7.3)
Presentation LUT	Known VOI outputs under IDENTITY, INVERSE, and an explicit Presentation LUT Sequence	Exact expected P-Value for each case (Chapter 8.2)
Palette Color	Small palette with known, distinguishable R/G/B values per index, including boundary indices	Exact RGB triplet per index, correct bit-depth scaling if LUT Bits per Entry $\neq 8$ (Chapter 10.3)
RGB	Synthetic 2×2 or larger images at both Planar Configuration = 0 and = 1	Exact per-pixel R/G/B recovered for both layouts (Chapter 11.1)
Planar Configuration	Same as RGB above, explicitly asserting layout-dependent byte offsets	No cross-contamination between channels
Multi-frame	Synthetic multi-frame dataset with distinguishable per-frame content (e.g. a frame-number pattern baked in)	Correct frame extracted for every frame index, correct stride math (Chapter 3.3, 16.7)
Shutters	Each Shutter Shape variant (rectangular, circular, polygonal) with known geometry and a known Shutter Presentation Value	Correct inside/outside classification per pixel; correct P-Value applied outside (Chapter 13.1)
Overlays	Synthetic Overlay Data bitmap with a known pattern	Correctly composited pattern, toggleable independent of Pixel Data (Chapter 13.2)
Spatial transformations	Known Image Orientation/Position/Spacing with hand-computed expected patient coordinates for several pixel positions	Exact match to hand-computed affine transform (Chapter 12.2, 16.9)

19.3 Comparing against a trusted reference implementation

L3 For each concern above, prefer numerical, per-pixel or per-value comparison against: (1) hand-computed expected values for small synthetic cases (most of the table above), (2) a well-established, widely-deployed reference toolkit's output on the same input for larger/real-world datasets, treating any disagreement as requiring root-cause investigation rather than assuming either implementation is automatically correct, and (3) DICOM-published test

datasets and conformance-oriented sample data where available, which exist specifically to exercise edge cases (unusual Bits Stored values, MONOCHROME1, various Photometric Interpretations) that ad hoc internally-generated test data may not think to cover.

19.4 Technical correctness versus clinical correctness

L3 **Technical correctness** means the implementation matches the DICOM standard's defined transformations exactly, verifiable by the numerical methodology above without any human judgment involved. **Clinical correctness** is the further, distinct claim that the resulting displayed image is fit for its intended diagnostic or clinical purpose — which additionally depends on display calibration (Chapter 8.4), viewing conditions, and appropriate preset/default selection, and which is properly validated through clinical/quality-assurance processes rather than software unit testing alone. A pipeline can be technically correct (every LUT, every stage, exactly standard-conformant) and still be clinically inappropriate if, for example, a non-calibrated consumer display is used for primary diagnostic reading, or if an institution's chosen default window preset is poorly suited to a particular study type. Conversely, two viewers producing visually similar images does not prove either is technically correct (19.1) — and does not, on its own, establish clinical correctness either, since a shared visual bug can persist across implementations that were validated only against each other rather than against the standard.

PRACTICAL TAKEAWAY FOR CODE REVIEW AND QA

Maintain a standing regression suite covering every row of the table in 19.2, keyed to specific chapter sections of this manual, and require any change touching the pixel pipeline to run against it before merge. Treat "the demo images look right" as necessary but explicitly insufficient sign-off.

20 The Engineer's Mental Model (Decision Flow)

Given an arbitrary DICOM image, here is the ordered sequence of questions that determines exactly what each pixel means and how it should ultimately be displayed. Every question below is answered in a specific earlier chapter, referenced inline.

1. **What is the Transfer Syntax?** Determines native vs. encapsulated Pixel Data and, if encapsulated, which codec must decode it before any further interpretation (Ch. 3.3).
2. **How is Pixel Data encoded?** Bytes-per-fragment/frame addressing for encapsulated data, or fixed-stride addressing for native data (Ch. 2.2, 3.3).
3. **What are the Image Pixel Module attributes?** Samples per Pixel, Rows, Columns, Bits Allocated, Bits Stored, High Bit, Pixel Representation, Planar Configuration, Number of Frames — read all of them before touching a single byte (Ch. 2).
4. **What is the Photometric Interpretation?** The hard branch point: MONOCHROME1/2, RGB, PALETTE COLOR, or a YBR variant (Ch. 4).
5. **Is the data grayscale or color?** Determines whether the Modality/VOI pipeline applies at all (grayscale, palette-index) or is bypassed (RGB, Ch. 11.4).
6. **Is there a Modality LUT?** Check for Modality LUT Sequence before assuming Rescale Slope/Intercept governs (Ch. 5.3).
7. **Is Rescale Slope/Intercept present?** If Modality LUT Sequence is absent, this is the mechanism; a fully-absent Modality LUT Module for a grayscale image not intended to carry one is unusual and should be treated as worth investigating, not silently defaulted (Ch. 5.2, 1.2 mandatory/conditional table).
8. **What is the modality/physical value?** The result of step 6/7 — the measurement-domain value, never itself a display value (Ch. 5.4).
9. **Is there a VOI LUT or Window Center/Width?** Determines the display-intensity-domain mapping; VOI LUT Sequence takes precedence if both present (Ch. 7.4).
10. **Is there a Presentation LUT?** Presentation LUT Shape or Sequence; defaults to IDENTITY if absent, which is still a meaningful, explicit choice, not "nothing happens" (Ch. 8.2).
11. **Is polarity involved?** Check both Photometric Interpretation (MONOCHROME1/2) and Presentation LUT Shape (IDENTITY/INVERSE) independently, and verify they compose correctly rather than being conflated into one flag (Ch. 8.3).
12. **Is color palette conversion required?** If PALETTE COLOR, expand through the three-channel LUT before any RGB-shaped code runs (Ch. 10).
13. **Are spatial transformations required?** Distinguish display-convenience flip/rotate from patient-space geometry derived from Image Orientation/Position/Spacing; never conflate them (Ch. 12).
14. **Are shutters present?** Check Shutter Shape and apply as a geometric compositing step using Shutter Presentation Value, downstream of Presentation LUT (Ch. 13.1).
15. **Are overlays/annotations/presentation states present?** Distinguish Overlay Data, Presentation-State graphic annotations, and burned-in annotation — each has different implications for toggability and de-identification (Ch. 13.2–13.4).
16. **What exactly reaches the rendering backend?** A fully-composed P-Value (or RGB triplet) array, plus any separately-tracked spatial transform and annotation layer — never a value array that has silently absorbed a spatial or compositing operation it cannot be distinguished from (Ch. 18.1).
17. **What does the monitor ultimately display?** P-Values pass through GSDF/display calibration — a property of the display chain, not something the pixel pipeline itself must always compute in software (Ch. 8.4–8.5).

USING THIS AS A CODE-REVIEW CHECKLIST

When reviewing a change to any rendering code path, walk this list and ask, for each question your change touches: "does this code answer the question by reading the actual DICOM attribute, or does it assume an answer?" Every assumption found this way is either a documented, deliberate simplification (acceptable, if documented and tested) or a latent bug from Chapter 15/17.

21 Glossary of Terms

Stored Value

The decoded numeric sample as it exists in Pixel Data, after byte assembly, Bits Stored masking, and sign extension, but before any Modality LUT transformation. Not itself clinically meaningful in general.

Pixel Value

An ambiguous, context-dependent term this manual avoids in favor of the more precise Stored Value / Modality Value / P-Value vocabulary; when encountered elsewhere, its meaning must be inferred from context.

Modality Value

The result of applying the Modality LUT (Rescale Slope/Intercept or Modality LUT Sequence) to a Stored Value — a physically or clinically meaningful quantity such as Hounsfield Units. The measurement domain.

Rescale

Shorthand for the linear Modality LUT mechanism, Rescale Slope and Rescale Intercept, applying $\text{Modality Value} = \text{Stored Value} \times \text{Slope} + \text{Intercept}$.

LUT (Lookup Table)

A tabulated input-to-output mapping, indexed by *input - First Input Value Mapped*, used identically by Modality LUT Sequence, VOI LUT Sequence, and Palette Color LUTs (Ch. 6).

Modality LUT

The transformation stage converting Stored Value to Modality Value, via Rescale Slope/Intercept or an explicit Modality LUT Sequence.

VOI (Value of Interest)

The transformation stage that selects and compresses a range of Modality Values into a bounded display-intensity range, via Window Center/Width or an explicit VOI LUT Sequence.

Presentation LUT

The transformation stage mapping VOI output to a polarity-correct, presentation-ready P-Value, via Presentation LUT Shape (IDENTITY/INVERSE) or an explicit Presentation LUT Sequence.

P-Value

The output of the Presentation LUT stage — a standardized, presentation-ready grayscale value intended for GSDF-aware display mapping.

GSDF (Grayscale Standard Display Function)

The DICOM PS3.14-defined perceptually-linearized mapping between a display driving quantity and target luminance, ensuring equal P-Value steps correspond to equal perceived brightness steps on a calibrated display.

Photometric Interpretation

The Image Pixel Module attribute declaring the semantic meaning of stored samples: MONOCHROME1, MONOCHROME2, PALETTE COLOR, RGB, YBR-family, and others.

MONOCHROME1

Single-sample grayscale Photometric Interpretation where minimum stored value displays toward white and maximum toward black (absent further inversion).

MONOCHROME2

Single-sample grayscale Photometric Interpretation where minimum stored value displays toward black and maximum toward white (absent further inversion); the more common convention.

PALETTE COLOR

Single-sample (Samples per Pixel = 1) Photometric Interpretation where the stored sample is an index into three parallel Red/Green/Blue lookup tables, not a color value itself.

RGB

Three-sample (Samples per Pixel = 3) Photometric Interpretation where stored samples directly are the red, green, and blue components.

Planar Configuration

The Image Pixel Module attribute governing whether multi-sample-per-pixel data (e.g. RGB) is stored interleaved (0) or as separate component planes (1).

Pixel Representation

The Image Pixel Module attribute declaring whether stored samples are unsigned (0) or signed, two's-complement (1) integers.

Bits Allocated

The physical container size, in bits, per stored sample (commonly 8 or 16).

Bits Stored

The number of meaningful bits within each allocated container, counted from the least significant bit.

High Bit

The bit position of the most significant meaningful bit within the container; equals Bits Stored – 1 for DICOM native encoding.

Window Center / Window Width

The parameters of a linear VOI transformation: Window Center is the Modality Value mapped to the middle of the output range, Window Width is the span of Modality Values mapped across the full output range.

Shutter

A geometric masking region (rectangular, circular, polygonal, or bitmap) excluding part of the displayed image, filled with Shutter Presentation Value; a compositing operation, not a value transformation.

Overlay

A separate 1-bit-per-pixel bitmap layer (Overlay Data) composited over the image at display time, distinct from and never mixed into Pixel Data.

Presentation State

A separate DICOM object (e.g. Grayscale Softcopy Presentation State) recording a complete, reproducible description of how a referenced image should be displayed — VOI, Presentation LUT, spatial transform, shutters, annotations — without modifying the underlying image.

Displayed Area

Presentation State attributes describing which sub-region of an image to display and at what sizing convention (true size, magnify, scale to fit).

Pixel Spacing

The physical row and column spacing, in millimeters, between adjacent pixel samples.

Image Position (Patient)

The patient-space (x, y, z) coordinate, in millimeters, of the center of the first (Row 0, Column 0) pixel.

Image Orientation (Patient)

The six direction-cosine values describing the patient-space direction of the image's row and column axes.

22 References and Further Reading

Primary standard (authoritative)

Part	Title	Relevance to this manual
PS3.3	Information Object Definitions	Image Pixel Module (C.7.6.3), Modality LUT Module, VOI LUT Module, Palette Color LUT Module, Presentation State IODs, Overlay Plane Module, Multi-frame Modules — the primary source for Chapters 2, 5–7, 9–13
PS3.4	Service Class Specifications	Background on the service-class context in which these objects are exchanged (storage, query/retrieve, print)
PS3.5	Data Structures and Semantics	Value Representations (VR), encoding rules for Decimal String (DS) and other attribute encodings referenced in Chapters 3 and 5
PS3.6	Data Dictionary	Canonical tag reference for every attribute cited throughout this manual
PS3.14	Grayscale Standard Display Function	Primary source for Chapter 8.4–8.5
PS3.16	Content Mapping Resource	Relevant background for structured/coded content occasionally referenced by Presentation State and annotation objects
Relevant Presentation State IOD definitions (PS3.3)	Grayscale Softcopy Presentation State and related color/pseudo-color/blending Presentation State IODs	Primary source for Chapter 13.3–13.4
Enhanced Multi-frame IOD and Functional Group definitions (PS3.3)	Enhanced CT/MR/XA and related Enhanced Multi-frame Image IODs	Primary source for per-frame functional-group variation discussed in Chapters 2.2 and 3.3

Framework documentation

For implementation-level detail on any specific version of Cornerstone or OHIF referenced conceptually in Chapter 14, consult each project's own current official documentation and source repository directly — library internals and supported-feature coverage evolve across releases, and this manual deliberately avoids asserting version-specific implementation claims that would go stale. Treat the architecture and division-of-responsibility framework in Chapter 14 as the durable content, and treat any specific library capability question as something to verify against the library's current documentation before relying on it.

How to use these references alongside this manual

RECOMMENDED PRACTICE

Where this manual states a mechanism (e.g. the exact linear VOI formula in Chapter 7.2, or LUT indexing in Chapter 6.3), treat the corresponding PS3.3 section as the normative source of truth and this manual as the implementation-oriented explanation of it. Where ambiguity exists between a vendor's actual output and this manual's description of standard behavior, consult PS3.3 directly, and treat any vendor-specific deviation as exactly that — a deviation to be identified and handled explicitly, never assumed to be the new normative behavior.

Closing note

This manual has repeatedly returned to one governing discipline: name every value space explicitly, never let a transformation's output be mistaken for another stage's input, and validate numerically rather than visually. If a reader takes away only that discipline and forgets every individual formula, they will still catch the majority of the failure modes catalogued in Chapter 15 in code review, because nearly all of them are, at root, a violation of exactly that discipline.